



Mars Science Laboratory

PLACES User Guide

Release: 2.0

Rev -B

JPL D-71121

MSL-586-3653

2011-08-19

Prepared by:

Bob Deen, MIPL/OPGS RDR Cognizant Engineer

Information contained in this document may be subject to U.S. export control laws and regulations (22 C.F.R. 120-130) and (15 C.F.R. 730-774). To the extent that information contained within is subject to U.S. export control laws and regulations, the recipient has the responsibility to obtain export licenses or other export authority as may be required before exporting such information to foreign countries or providing access to foreign nationals.



Jet Propulsion Laboratory

California Institute of Technology

Pasadena, California

1. Introduction.....	5
1.1. What is Localization	5
1.2. Purpose of PLACES	5
1.2.1. High-Level Requirements.....	5
1.3. What PLACES is Not.....	6
2. Background	7
2.1. Coordinate Frames and References	7
2.1.1. Rover (Nav) Frame.....	7
2.1.2. Site Frame	7
2.1.3. Local Level Frame.....	8
2.1.4. Orbital Frames.....	8
2.1.5. Reference Frames	8
2.2. Rover Motion Counter (RMC)	9
2.2.1. Site Index.....	9
2.2.2. Drive Index.....	9
2.2.3. Pose Index.....	9
2.2.4. Mechanism Indices.....	9
2.2.5. RMC as a Clock.....	10
2.3. Rover Behavior during Drives and Arm Moves.....	10
2.3.1. Driving	10
2.3.2. Arm Operations.....	10
2.4. Saved Frames	11
2.5. Localization Methods	11
2.5.1. Onboard Methods	11
2.5.2. Ground-Based Methods.....	11
3. PLACES Basics	13
3.1. Immutability of Results	13
3.2. Frames.....	13
3.3. Solutions.....	14
3.4. Views.....	14
3.4.1. Open and Closed Views	15
3.4.2. Standard Views	15
3.5. Positions vs. Orientations.....	16
4. Reading from PLACES.....	17
4.1. Primary Query.....	17
4.2. Rover Frame Query Forms	18
4.2.1. 2-Element RMC (site, drive)	18
4.2.2. 3-Element RMC (site, drive, pose)	18
4.2.3. Full 10-Element RMC.....	18
4.2.4. Time (SCLK)	18
4.2.5. Full 10-Element RMC Plus SCLK.....	19
4.3. Using the Stable RMC Subset.....	19
4.3.1. Queries Using the RMC Pose Counter	19
4.4. Using Dynamic (in-motion) RMC's.....	20
4.4.1. Non-Immutability of Results.....	21
4.5. Unsolicited Motion	21
4.5.1. Mechanism-Induced	22
4.5.2. External Events.....	22

4.6.	Access to Raw Data.....	22
4.7.	SCLK Lookup	22
4.8.	Saved Frame Lookup	23
4.9.	Summary of Immutability	23
4.10.	XML Summary Files and PDS Archive	23
5.	Creating New Localizations.....	25
5.1.	Steps Required	25
5.2.	Metadata.....	25
5.3.	Solution ID's	25
5.4.	RMC's	26
5.5.	Solutions.....	26
5.6.	Using the Derived-from Frame	26
5.6.1.	Rules of Thumb.....	28
5.6.2.	Derived-from and Sites.....	29
5.6.3.	Derived-from and Views.....	30
5.6.4.	Derived-from and Orientations	31
5.6.5.	Derived-from and Dynamic RMC's	31
5.7.	Dynamic RMC's and Unsolicited Moves.....	31
5.8.	Creating Views.....	32
5.8.1.	Parent Views	32
5.8.2.	Cloning vs. Deep Copy.....	33
5.9.	Groups and Users	33
6.	Detailed Design Topics	35
6.1.	Are Sites Parallel?	35
6.2.	Coordinate Transform Algorithm.....	35
6.2.1.	Determining the Derived-from Offset.....	36
6.2.2.	Calculating a Solution in its Own Reference Frame	36
6.2.3.	Changing Reference Frames.....	37
6.3.	Frames and Subframes	37
6.4.	Why "telemetry" is Special	38
6.5.	Mapping of Orbital Images	38
6.6.	Back Doors.....	39
6.7.	Sources of Telemetry Data	40
6.8.	Approvals.....	40
7.	PLACES and MER.....	41
7.1.	RMC Differences.....	41
7.2.	Lack of Pose Index.....	41
7.3.	Queries.....	42
7.4.	Immutability	42
7.5.	Dynamic Drives.....	42
7.6.	Simulating Pose.....	42
8.	How to Access PLACES	44
8.1.	ReST Web Service	44
8.2.	Authorization and Authentication.....	44
8.3.	PLACES Resources.....	44
8.4.	Get and Put Operations	45
8.5.	Base URLs.....	45

9. URLs and Query Results	46
9.1. Query Element Syntax.....	46
9.2. GET Queries.....	47
9.3. Get Query Response Types.....	50
9.4. Put Operations	54
9.5. Put XML Payloads	57
10. Use Cases.....	59
10.1. What was the rover's position at RMC (5,12)?	59
10.2. How does Joe think the rover was oriented at RMC (5,12)?.....	59
10.3. What Views have you created?.....	59
10.4. Creating a Traverse Map	59
10.5. Mosaic Two Images from Different Positions	60
10.6. Update Target with New Localizations.....	60
10.7. Point the Camera at a Target	61
10.8. Create New Localization in Existing View.....	61
10.9. Create New View with an Update.....	62
11. Glossary.....	64
12. Acronym List.....	67
13. TBD list.....	68

1. Introduction

This document describes the use and design of the PLACES (Position Localization and Attitude Correction Estimate Storage) database. The first few sections provide conceptual context; what is PLACES and why and how you should use it. This is presented in increasing order of complexity, from simply using the localizations to creating them and detailed design topics. The last sections contain the mechanics of how to talk to the database, syntax for the various queries and data uploads, and what results to expect back.

Although this document is MSL-specific, PLACES can also be used for MER and other rover missions (see Section 7). Each mission would have its own separate database.

1.1. *What is Localization*

Rover localization is the process of determining the position (or location) and orientation (or attitude) of the rover at any given time. This process is fraught with error and the true value is never known; only estimates (or “solutions”) of the correct value exist. These estimates will differ depending on how they were produced, and may sometimes conflict with each other. Yet each estimate is valid for its intended purpose by its intended audience.

Localization solutions are produced both onboard the rover, and by ground analysis of data.

1.2. *Purpose of PLACES*

The purpose of PLACES is to manage all these solutions, from users throughout the operations and science teams as well as those produced by the rover itself, and provide a coherent view of them across the Project.

Thus, PLACES is a “one-stop shopping” source for localizations. It applies no value judgments to the localizations; each is potentially as useful as any other. The user specifies which set of solutions to use (the View mechanism), and the database supplies those solutions in an easy-to-use manner.

The database is accessed via a ReST-style web service, which is a common style of implementing web services. This means that each function is accessed via a URL and all queries and updates are atomic. All services of the database can be exercised using a simple Web browser, but are also available via command-line tools and programmatic requests. Web pages wrapped around these URL queries show the state of the database and allow simple browsing of its contents, as well as form-based updates.

1.2.1. High-Level Requirements

The requirements for PLACES are:

- Provide a single Project-wide source for localization solutions.
- Accept and store localization inputs from any team member.
- Provide access to all submitted raw localization data.

- Provide a simple and convenient mechanism to answer common queries.
- Provide views on the localization data that present a single position/orientation value for each RMC.
- Guarantee immutability of results, so the same query always returns the same answer (subject to some restrictions).
- Provide access via Web service (ReST-style queries), Web page, programmatic request, and summary XML files.

1.3. *What PLACES is Not*

PLACES deals with rover localizations. This includes the position and orientation of the rover, the Sites, and the orbital images used for navigation. It can do this to very high fidelity, depending on the telemetry sent.

However, PLACES does *not* store the locations or pointings of mechanisms on the rover, such as the arm, mast, antenna, mobility system (rocker bogies), or any part of the sample-handling chain. It tells you where the rover is, not any of its pieces.

Thus it is not a complete frame manager. The reason for this is that positions of mechanisms are not generally updated on the ground, so the telemetered value is sufficient and there's no need to store additional solutions. An exception to that is the camera mast pointing, which is adjusted on the ground for mosaic correction. However, there are existing mechanisms for that; adding the additional complication to PLACES was not deemed worthwhile.

PLACES also does not apply value judgments to the localization solutions - i.e. deciding which one is better. That is up to the user to decide. It merely maintains the solutions and translates between them as requested.

2. Background

This section provides some background on the MSL mission that are useful for understanding PLACES.

2.1. *Coordinate Frames and References*

A variety of coordinate frames are used on the MSL mission; there are literally dozens. Of these, however, only four are relevant to rover localizations.

2.1.1. Rover (Nav) Frame

The Rover Navigation Frame (usually called just Rover frame in this document) is attached to the rover, with the origin fixed at a point on the nominal ground, between the middle wheels. The +X axis points out the front of the rover, with +Z pointing away from the belly (“down”), and +Y completing the right-handed system. Thus any part of the rover that doesn’t move has a fixed coordinate in the Rover frame. However, the entire frame “moves” relative to the surface during drives, and thus the numeric coordinates of rocks as measured in a Rover frame change.

Thus there exist “instances” of the Rover frame, indexed by the RMC (see Section 2.2). Each instance is valid during a certain period of time, during which the coordinates of surface objects in Rover frame do not change.

The origin and orientation of these Rover frame instances describe the rover’s location throughout time. Refining knowledge of these origins is the primary goal of rover localization.

Note that there exists a Rover Mechanical frame, which is closely related to Rover Navigation but with a constant offset. It is not used in the PLACES system.

Because the Rover frame is attached to the rover, the position and attitude of the rover itself at any given time is the same as the position and attitude of the rover frame at the same time. The concepts of rover pose and Rover Frame pose are thus synonymous, and the terms are used interchangeably in this document.

2.1.2. Site Frame

Because the coordinates of surface objects measured in Rover frame change whenever the rover drives, it is advantageous to have a frame that is fixed to the surface.

However, on-board rover position estimation is rather imprecise; errors of 10% are common, more if the wheels slip excessively. Visual Odometry (visodom) reduces this error significantly, but does not eliminate it. If only one surface-fixed frame were available, accumulation of error over the mission would be significant. Therefore multiple Site frames are used, each fixed to the surface in a certain place.

The Site frame thus defines the general vicinity of the rover. Most operations are carried out relative to the current Site frame. When the rover moves too far away from the current Site origin, a new Site is declared, with a new origin, and the Site index is incremented.

Sites allow isolation of localization errors. If it is determined that a particular drive was improperly localized early in the mission, only the origins of subsequent Site frames need to be corrected. All data within a subsequent Site is unaffected, because they are defined relative to their own Site origin.

Site frames are defined with +X pointing north, +Z pointing down according to the local gravity vector, and +Y completing the right-handed-system. The origin coincides with the Rover (Nav) frame at the time the site is declared.

2.1.3. Local Level Frame

Local Level frame is a combination of Site and Rover frames. It shares an origin with the Rover frame, thus moving with the rover and being instanced via the RMC. However, it shares the orientation of the Site frame, pointing north/down. The Local Level frame can always be easily computed given both the Site and Rover frames, therefore no explicit information about Local Level frames is stored in the database. However, Derived-from frames (see Section 5.6) are commonly expressed in Local Level.

2.1.4. Orbital Frames

The final relevant frame type is Orbital frames. These are defined by orbital imagery, which must be in a rectangular projection. The details of orbital mapping are described in Section 6.5, but coordinates are defined in the same XYZ system as for Site and Rover frames.

2.1.5. Reference Frames

All numeric values, whether a position or orientation, must be expressed relative to some other frame. It makes no sense to say the rover is at (10.5, 5.2, 0.1) without also saying how to interpret those numbers. Thus every coordinate or rotation must have a Reference frame, in which the values are expressed.

Generally Rover frames are expressed in the Site frame sharing the same Site index. Site frames are generally expressed in terms of the preceding Site frame. However, these guidelines are not absolute, and any frame can be expressed in terms of any other, as long as a tree structure is maintained (i.e. no circular references).

The root of the tree for PLACES is the Root frame, which is defined as the landing site for both MSL and MER. This frame has no numeric location, no solutions, exists implicitly in every view, and is identical across all views. Orbital frames and the landing site (Site 1 for MSL, Site 0 for MER) are all defined in terms of Root. The landing site should have a 0 offset and identity quaternion from root.

The Root (landing site) does of course have a location, expressed as a latitude/longitude on Mars. The location of Mars is expressed in a solar system inertial frame, and so on. But these are outside the scope of PLACES.

2.2. Rover Motion Counter (RMC)

The Rover Motion Counter (RMC) is a set of ten indices whose values serve to uniquely identify every location the rover has visited, and thus every instance of the Rover Frame. It also serves to indicate when any mechanism on the vehicle has moved.

The ten indices, in order, are:

Site, Drive, Pose, Arm, CHIMRA, Drill, RSM, HGA, DRT, IC

2.2.1. Site Index

The Site index defines which instance of the Site frame is relevant for this RMC. Whenever the Site frame is incremented, all the other RMC values are set to 0. Declaring a new Site frame thus resets all motions and creates a new local area in which to work.

Unlike the other indices, there is no meaning to odd or even values. Also unlike other indices, Sites start with 1 (the landing site), not 0.

2.2.2. Drive Index

The Drive index (sometimes called Position) increments whenever the rover drives or otherwise moves its wheels (e.g. trenching or steering). The value is odd while the wheels are actually moving, and even otherwise. Incrementing Drive sets all the other indices (except Site) to 0. Exception: if a mechanism index is odd, indicating it is moving during the drive, then that index is not reset to 0. This should be an unusual case, however. The Drive index is the one of most interest for most localization activities.

2.2.3. Pose Index

The Pose index indicates a change in the rover's knowledge of its position or orientation. Unlike the other indices, it does not indicate that anything actually moved; only that the pose knowledge has changed. This may be due to running visodom, doing a sun find, reading the IMU's (Inertial Measurement Units), or receiving an explicit ground command to update the pose knowledge.

Pose will be odd while an update is actually being made (including extended IMU updates), even otherwise. Incrementing Pose has no effect on other indices.

2.2.4. Mechanism Indices

The remaining RMC indices (Arm, CHIMRA, Drill, RSM, HGA, DRT, IC) are associated with mechanisms on the rover. They increment to an odd number when the mechanism is moving, and again to an even number once the mechanism stops. More than one mechanism may be moving at the same time.

The MSL arm has the strength and mass to move the rover (in tilt, primarily) when it is used. Thus the fact that the arm moved, as reflected in the RMC, means that the rover's physical pose may have changed, even if its knowledge has not. This is sometimes relevant and sometimes not, but is the primary reason for maintaining the mechanism RMC indices. The other mechanisms could in theory similarly move the rover, but are unlikely to in practice.

2.2.5. RMC as a Clock

The RMC can be thought of as a clock, albeit one that ticks at irregular intervals. It measures time not as a duration, but in terms of motions.

Given any two valid RMC's, one can determine unambiguously which occurred earlier in time, because the counters only count upwards (except when reset to 0 by incrementing Site or Drive). If the Sites are different, the lowest is earlier. If the same, then the lower Drive is earlier. If those are the same, then consider the remaining indices. If any are lower, then the entire RMC is earlier. Note that it is not valid for some of these indices to be lower and others higher - that indicates the RMC value was corrupted, possibly during transmission.

2.3. *Rover Behavior during Drives and Arm Moves*

The interaction of the RMC with driving and arm motion is relevant to localization, so it is described here.

2.3.1. Driving

Before a drive starts, the Drive index is incremented to an odd value, where it stays throughout the drive. Once the drive ends, the index is incremented to even. During the drive, the rover position and orientation are computed and propagated by the flight software at an 8Hz rate. This means that the same RMC, with an odd-valued Drive index, can actually refer to many different places along the drive.

For this reason, odd-drive RMC's (called dynamic RMC's) are insufficient to specify a location; actual time must be used instead (see Section 4.2). However, very few other activities that require localization (such as imaging) happen during a drive, so this is rarely an issue. To handle the instances where it is (such as taking video while driving), PLACES will store all pose updates it sees in telemetry during a drive, for use with time-based queries.

Note that the Pose index does not increment during the drive, despite the fact that pose knowledge is changing.

If visual odometry is run to correct the rover's position after a drive, the drive index is first incremented to even. Pictures are then acquired and analyzed onboard. Once a visodom solution is computed, the Pose index is incremented to odd, the value is stored, and Pose is incremented to even again. If the visodom image is downlinked, it will contain the position computed during the drive, not the visodom solution.

2.3.2. Arm Operations

Because the arm has the possibility of moving (tilting) the rover, special precautions may be taken during arm operations. If enabled, the Pose counter is first incremented to odd. Arm operations then commence. While Pose is odd, the rover attitude is continually monitored and updated at an 8Hz rate. Once the arm motion stops, Pose is incremented to even and the updates stop. Note that the Arm index would have incremented as well due to its motion, along with potentially those of other mechanisms.

This is similar to the behavior during driving, and RMC's with odd Pose values are also called dynamic RMC's. However, arm operations often take longer, and there are more likely to be other activities occurring at the same time. Thus it is more likely to see an odd Pose value in other data, such as imagery. Also, the rover position is not updated during arm ops; only the orientation is. PLACES will again store orientations seen in telemetry during moves, for use with time-based queries.

Not all arm ops will monitor the pose. If not, attitude knowledge will not change (even if the rover moves), because the Pose index did not change.

2.4. *Saved Frames*

The rover has the concept of Saved Frames. This means that the Rover and Site frames at a specific RMC are saved onboard for later use. Saved frames have a single index, which can be positive (an absolute saved frame number) or negative (relative to the most recent saved frame).

A Saved frame is really nothing more than a shorthand to specify the entire 10-element RMC value that was saved. For that reason, Saved frames play no role in PLACES; the full RMC will be used instead. However, as a convenience, PLACES will maintain a table mapping saved frame numbers to their corresponding RMC (see Section 4.8).

2.5. *Localization Methods*

Localizations can come from many sources, using many methods. The following list is not comprehensive, but is intended to illustrate what is meant by localization. Note that all onboard methods increment either the Drive or Pose indices.

2.5.1. Onboard Methods

- Wheel rotation integration. This is the primary method for the rover to compute its position onboard, and happens during all drives.
- Visual odometry (visodom). This is a more accurate onboard method for position estimation, which analyzes images before and after a drive to compute how the rover moved.
- IMU. Onboard inertial measurement units as well as gravity sensors (accelerometers) allow continuous estimation of orientation.
- Sun find. A more accurate method of estimating orientation on board, which takes images of the sun to compute how the camera, and thus the rover, is oriented.

2.5.2. Ground-Based Methods

- Image analysis. This is the process of analyzing images taken before and after a drive (or series of drives) to refine the pose. Similar to visodom, but likely to be more accurate in ground processing.

- Target updates. An interactive process by which a predefined target is adjusted to match new imagery, thus creating a pose update.
- Orbital fix. Determining where the rover is in an orbital image, either by seeing it directly or by triangulating from landmarks. Can also be used for orientation, although this is not common.
- Radio tracking. The Navigation team can use UHF two-way Doppler tracking of the rover to estimate rover position in the Mars inertial system, which can be converted to the Mars body-fixed or orbital-image reference frames used by PLACES.
- Besting. The process of taking a set of localization solutions and combining them into one “best” solution by considering their error metrics.
- Bundle adjustment. A process that gathers together lots of information and runs a global minimization function to adjust many results simultaneously in order to obtain the best overall solution. More refined than besting since it typically operates with raw tiepoint or image correspondence information rather than prior pose estimates.

3. PLACES Basics

This section covers the fundamental concepts of PLACES.

3.1. *Immutability of Results*

One of the top-level requirements of PLACES is to guarantee immutability of results, except in certain defined cases. That means that an identical query will return the same answer throughout time. This is critical for tactical planning, in order that ongoing localization activity does not disrupt target definitions.

Every query to PLACES involving coordinate transforms returns a flag that indicates whether or not the result is immutable. Cases where the results are mutable are explicitly called out in this document; also see Section 4.9.

Metadata (for example, who created a solution) which does not affect any numeric results is *not* immutable; it can be changed at any time. It is also nullable, so it can be filled in later or left out.

3.2. *Frames*

A “frame” (coordinate frame) is what is defined by PLACES. Frames are identified by a frame type and an instance, which may be an RMC, time value, or index. Note that a frame is not the same as a solution. Frames unambiguously exist; it’s their location and orientation that is uncertain.

PLACES defines five frame types:

- Site
- Rover
- Orbital
- Local_Level
- Root

All of these except root correspond to the coordinate frames described in Section 2.1. It is the combination of frame type and associated instance identifier (RMC, time, or index) that specifies what is being defined. For example, a Site frame requires an index, thus Site 5 is a frame. So is Rover (5,12,2,6,100,8,0,0,12,2), Rover at SCLK 410455612, Rover (5,12), or Orbital image 3. Local_Level frames are not stored in the database, but can appear in some queries. Internally a query for Local_Level position is transformed into a query for Rover position, and a query for Local_Level orientation is transformed into a query for Site orientation. Similarly, use of Local Level as a reference frame is transformed to a Rover position reference or Site orientation reference.

The Root frame is used as the single reference from which all others are ultimately derived, and corresponds to the landing site. See Section 2.1.5

Internally, the frame type is converted to an integer and used like a 0th index in the RMC. This should generally be transparent to the user.

3.3. Solutions

Solutions are the currency of PLACES. Localization is not an exact science, and multiple estimates of the rover pose from multiple places can be simultaneously valid for different reasons. Each of these estimates is called a “solution”.

A solution is a single estimate of position or orientation for a single frame, relative to some reference frame. Once a solution is defined, it may never be changed for any reason.

Each solution is named by a solution ID. Solution ID’s may be shared among multiple solutions, but the pair (Frame,ID) must be unique. One frame can have several solution ID’s associated with it, and one solution ID can be re-used for multiple frames.

What a solution really does is to define the relationship between two frames - the one that’s part of the solution name, and the one named by the Derived-from frame (not the reference frame; see Section 5.6). If the Derived-from frame moves relative to some third frame, the solution’s frame may move as well relative to that third frame - but the relationship between the two is fixed. For example, if someone redefines a Site frame, the Rover frames inside it retain the same relationship to the Site, but may have a different relationship to an earlier Site. In other words, solutions exist in isolation.

3.4. Views

Working with multiple, possibly contradictory, solutions for the same frame is extremely difficult. In addition, in order to make use of the information contained in a solution, it must generally be placed into a broader context, containing relationships to other frames.

For these reasons, PLACES has a concept of named Views. A View is simply a way of looking at the database that assigns one and only one solution to each frame. Which solution that is, will depend on how the particular View is defined.

Thus, a View is simply a list that specifies which one solution ID to use for each frame. It’s a shorthand, relieving the user of the burden of specifying a huge list of solution ID’s every time they want to use the database.

Views are generally defined in terms of a parent View, with changes applied. For example, if Joe likes Sally’s View but disagrees with a couple of rover positions, Joe could create a new View with Sally’s as a parent, and then redefine the positions he disagrees with.

Views are thus logically dense (containing a solution for every frame), but are physically sparse (containing just those solutions that differ from the parent).

Views can be created by any user, and users can control via the Groups mechanism who is able to modify their view. However, all views are public - there is no restriction on anyone reading a View.

It is expected that many views will contain the same base name with a counter. This counter may be a Sol number (as with `best_tactical_nnnn`), or simply a version number. For this reason, a special syntax of “^” is allowed when referring to view names, which says to use the highest-numbered view that matches the name (for example, “`best_tactical_^`”). The caret also works for solution ID’s and RMC components. However, use of the caret makes the result mutable, since the highest number may change (although if this is the only reason the query is mutable, it can be “resolved” to an immutable query; see Section 4.9).

3.4.1. Open and Closed Views

One of the primary ways that immutable results are implemented in PLACES is via the concept of open and closed views.

An open View is mutable - solutions can be added or removed at will, and there is no expectation of immutability. All views are open when they are created; this allows the view to be constructed. Identical queries made to an open View could return different results as solutions are added or deleted.

A closed View is immutable - it can no longer be changed, and identical queries will always return the same result.

However, views are not simply “closed”. They are closed up to (and including) a point, defined via an RMC value, and are open after that point. Queries made before (or equal to) the close point are immutable; those after are not. The close point can be moved later in time (to close more of the view), but not earlier.

Immutability is discussed further in Section 4.9. In order to eliminate confusion, all query results involving coordinate transforms contain a flag saying whether the result is mutable or not.

3.4.2. Standard Views

Views can be created or used by any user. However, a few special views will be maintained by the system and can be relied upon to exist.

- **telemetry** - This is a single View that contains all solutions named “telemetry”, and only those solutions - in other words, every piece of localization data downlinked by the rover. There will generally be a telemetry solution for every available frame, although it is possible for there to be enough information to compute a frame on the ground for which there are no telemetered values (such frames would have no entry in the “telemetry” view of course). This is an important view since serves as the root of the view tree, but it should generally not be used directly, since visodom solutions will not be included. “tactical” should be used instead. One exception to this is rover commanding; the rover only knows what’s in “telemetry” so coordinates in commands should be translated to a “telemetry” solution. The 3-element form should be used in preference to the 2-element for this case.
- **tactical** - A single View that contains the quick-response solutions to be used for tactical planning. This is generally the same as the “telemetry” view, except that visodom results will be available using the simple (Site, Drive) RMC subset queries. It is possible, however,

for extremely quick-response ground solutions to be included in “tactical”, at the discretion of the tactical ops team. At some point after the downlink ends but before the planning cycle starts, this view will be closed up to the latest available RMC, guaranteeing immutability throughout the planning cycle.

- `best_tactical_nnnn` - A View that contains the best tactically-relevant solutions as of a given Sol. Because a new View is created for every Sol, it is possible to incorporate localization updates from prior Sols (unlike “tactical”). Thus, this View should contain much more accurate localizations than the “tactical” view. The View should be created at the same time “tactical” is closed. So it will typically contain the same solutions as “tactical” for the current Sol, but with better data for earlier Sols. Once “`best_tactical_nnnn`” is closed, it is closed permanently, not just to “now”, like “tactical” is. This allows version `n` to use `n-1` as a parent yet still get current “tactical” updates and apply overrides.

It is anticipated that the science team will define similar team-wide standard views, as well as those defined by each team member as needed.

3.5. *Positions vs. Orientations*

Positions (offsets in XYZ space) and orientations (attitude of the rover) are treated completely separately by PLACES. Within one View, a given frame can have one solution ID for the position and a different one for the orientation. Although both are needed for a full coordinate frame transform, they are very commonly derived separately, so a full separation makes it easier to work with them. This document often talks about positions for convenience, especially in terms of coordinate transforms. In most cases, the concept applies equally well to orientations (where an offset maps to a rotation and adding offsets maps to compositing rotations).

4. Reading from PLACES

This section discusses how to get information out of PLACES.

4.1. *Primary Query*

There is one primary query that is the main purpose of PLACES:

- What is the position or orientation of a frame (such as the Rover frame) at a certain time, measured in some other frame, according to a specific person or to the rover itself?

The inputs to this query are thus:

1. Whether to return position, orientation, or both
2. The frame in question (type and instance)
3. The reference frame desired for the answers (often a Site frame)
4. The view name

The output consists of:

1. The numeric position and/or orientation (as a quaternion)
2. The reference frame and view in which the numbers are expressed
3. Whether or not the result is immutable

The reference frame and view are echoed in case the “^” (last) or “@” (first) syntax are used; these are resolved to the actual names and index numbers in the output.

Behind the scenes, PLACES may have to do considerable computation to respond to this query, as it follows all of the Derived-from frames to build a consistent picture of the frame’s pose, as well as converting reference frames according to the views they are expressed in. The detailed algorithm is described in Section 6.2. This allows one to, say, change the location of site 3 relative to site 2 in their view, and have that change propagate throughout the entire frame tree as appropriate.

There is also a transform operation, where one can supply a position or direction (unit) vector and view in addition to the above, and receive the same point or vector back, transformed to the given reference frame and view. This can be used to convert coordinates of objects, such as a rock, between frames and views.

The inputs to the transform query are thus:

1. Numeric position or unit direction vector
2. Frame (type and instance) and View in which the numbers are expressed

3. Desired Frame and View

The output consists of:

1. Transformed numeric position or unit direction vector
2. Reference frame in which the numbers are expressed
3. Whether or not the result is immutable

Beyond this query, the most common operation will likely be to ask for the available views, and metadata about them, in order to select the proper one to use.

4.2. *Rover Frame Query Forms*

Site and Orbital frame types require only a single integer to identify their instance. Unfortunately the Rover frame is not so simple. Note that Local Level works the same as Rover frame in this section.

Queries for rover frame instances can take five forms, each with its own use case. They are explained more fully in the sections to follow.

4.2.1. 2-Element RMC (site, drive)

This is the simplest form, and is sufficient for most uses. If all you care about is rover driving, then this is the form you should be using. Most mobility operations look at site and drive only; the rest of the RMC is not relevant. If the specific site and drive is not present, the query results in an error. This query form will not see unsolicited moves, however.

4.2.2. 3-Element RMC (site, drive, pose)

This is used in certain special queries involving the Pose as a solution ID (see Section 4.3.1) but is otherwise similar to the 2-element RMC query.

4.2.3. Full 10-Element RMC

The full 10-element RMC is only needed if you want to track unsolicited moves - when the rover moves unexpectedly due to arm or other mechanism motion. If the exact RMC specified is not available, the query returns information for the closest RMC before the specified one that shares the same site and drive.

4.2.4. Time (SCLK)

The Rover frame instance can be identified by time rather than RMC. This is usually only of interest for dynamic RMC's, where the rover is actually moving. If the exact time specified is not available, the query returns information from the closest entry available that shares the same site and drive, which may be before or after the specified time (note the difference from the 10-element RMC case, which is before only). Time-based queries are generally not immutable, unless you hit the time exactly. Since time is stored as a double-precision float (in order to accommodate

8Hz updates), exact hits are difficult and should not be expected. Note that time can be used to query any RMC, not just dynamic ones, but this is discouraged for efficiency reasons.

4.2.5. Full 10-Element RMC Plus SCLK

This combines the 10-element RMC and SCLK queries into one. It is sometimes the most convenient form to use when you want to get the most accurate information available for a specific image or other data set. By supplying both, you do not have to think about unsolicited moves or dynamic RMC's; simply look at the result to determine if it is immutable or not. The system will first do a lookup by RMC. If this results in a stable RMC with no subframes (unsolicited moves), then that result is returned and the SCLK is ignored. If the RMC is dynamic or contains subframes, then the SCLK is used if necessary to find the correct frame to use.

The fact that the SCLK is ignored if not needed is important; it significantly raises the odds of obtaining an immutable solution. Thus this is not the same as a purely time-based query.

It is assumed that the SCLK is within the time interval the given RMC was active. If not, it will find either the earliest or latest frame available for that RMC.

4.3. *Using the Stable RMC Subset*

A “stable” RMC is defined as one where the Drive and Pose indices are both even. This means the rover is neither driving, nor updating its position. Thus, the position knowledge is “stable”.

Stable RMC's are far easier to work with than dynamic ones, and it is recommended to use them whenever possible.

In the “telemetry” view, the mechanism values are discarded for stable RMC's, because moving mechanisms can have no possible impact on the pose knowledge. That means any query of “telemetry” using a stable 10-element RMC will be equivalent to the corresponding 3-element query.

Note, however, that views other than “telemetry” can have 10-element RMC's within a stable RMC whose value differs. This would be caused by someone on the ground doing a localization of an unsolicited move (see Section 4.5).

4.3.1. Queries Using the RMC Pose Counter

The Pose counter of the RMC is a special case. Fundamentally, a Pose update is entirely analogous to a solution ID. An even value represents a change in pose knowledge, just as a new solution ID does. The difference is that the rover knows about Pose updates (it knows nothing of other solutions), and the Pose number is part of the RMC. This necessitates some special handling.

Odd values of Pose indicate a dynamic RMC situation. For these, Pose is simply part of the RMC “clock” and is not handled specially. The rest of this section covers the even case.

When a telemetered, stable RMC is entered in the database, the mechanism indices are discarded, leaving only Site, Drive, and Pose. The location/orientation are stored in a solution named “pose_*p*”, where “*p*” is the Pose number, attached to the RMC (Site, Drive). If Pose is 0, then the

location/orientation are additionally stored in a “telemetry” solution for (Site, Drive). The “pose_p” solution can be added to any view, just like any other solution. Note that users cannot create their own “pose_p” solutions; the name is reserved for system use just as “telemetry” is. The “pose_p” solutions are in reality an extension of “telemetry”.

Thus, standard two-element queries of the “telemetry” view will return the value for pose 0. If pose 0 is not available, this will return an error, as with any other 2- or 3-element query when the value is not available.

An important result of this rule is that *(Site, Drive) queries to “telemetry” will **not** return the results of visual odometry!* This is because visodom results cause a Pose update, meaning Pose is non-0. This is the primary purpose for the “tactical” view - it assigns the latest available “pose_p” solution (say, “pose_2”) to the (Site, Drive) RMC, meaning that simple (Site, Drive) queries do indeed return visodom results.

It is for this reason that the “tactical” view should be used in preference to the “telemetry” view in almost all situations (except commanding the rover; see Section 3.4.2).

Three-element queries (Site, Drive, Pose) are handled specially. These are converted internally to a query for RMC (Site, Drive) with the solution ID “pose_p”. This happens regardless of what View is in use, or what solution ID is assigned to that (Site, Drive) by the View. A 3-element query always returns the specific pose as seen in telemetry. If the Pose doesn’t exist for that (Site, Drive), an error is returned.

The (Site, Drive, Pose) query serves as a shorthand to be able to see an arbitrary Pose no matter what View you are in. It also makes Pose look like it is part of the “telemetry” view, and enables collapse of stable 10-element RMC queries of “telemetry” into the equivalent 3-element one. It is useful when translating rover commands to the “telemetry” view.

As a further shorthand, a caret (^) for the Pose value in a query (e.g. (5,12,^)) returns the latest Pose index for that (Site, Drive). Because new Pose’s could be added at any time, use of the “^” makes the query mutable. (The “^” syntax can actually be used for any index, not just Pose). For completeness, an at sign (@) can be used to indicate the lowest (earliest) number, which could also be mutable if it does not resolve to 0. If the “^” or “@” is the only reason the query is mutable, this can be resolved to an immutable query; see Section 4.9.

Because (Site, Drive, Pose) frames do not actually exist in the database, they cannot be the basis for other solutions. It does not make sense to create a new solution for a Pose any more than it does for any other solution - both are solutions for the underling (Site, Drive) RMC.

4.4. Using Dynamic (in-motion) RMC’s

Dynamic RMC’s are defined as those with an odd value for either Site or Pose (or both). They indicate that the rover is actively driving, or in the middle of updating its pose knowledge (possibly during an extended period of arm motion). This behavior was described in Section 2.3.

Dynamic RMC's act very different in PLACES than stable ones. Primarily, the rover's pose is continually changing throughout the period the Drive or Pose indices are odd. This means that there are many distinct frames that share the same RMC value. In theory this could mean infinitely many frames, but since the pose is sampled at 8Hz onboard, that provides an upper bound.

Because of this, the RMC is not sufficient to describe which frame you want. Instead, time must be used: the spacecraft clock (SCLK). The RMC *can* be used, and will return the first available time that matches the RMC (or revert to the nearest RMC before the given one if there is no match). But that provides no insight into the motion itself.

Dynamic RMC's can be localized and provided with a solution if desired. However, this is unlikely to be very useful, since the localization applies to a single instant in time.

Note that dynamic RMC's due to driving can update both position and orientation. However, dynamic RMC's due to arm operations can only change orientation - position, although it may physically change, will not be updated by the FSW during arm operations. Not all Pose updates are arm ops though; those due to visodom or ground commands can update position as well as orientation.

4.4.1. Non-Immutability of Results

A query based on time is inherently mutable, because the database does not contain every possible time stamp - only those that were downlinked. If the given time exactly matches one in the database, then the value is immutable, because we have specific information for that instant. However, this is unlikely since time is stored as a double-precision float, in order to accommodate the potential 8Hz updates.

If the exact time is not found, then the closest available time (either before or after) is returned. The value should be considered mutable, because there is no way to know if another closer time stamp, unknown to the database at the time of the query, might become available later.

If the exact time is not found, but unambiguously translates to a stable RMC that would otherwise be immutable, then the query is considered immutable. Unambiguous in this case means that the end time of the stable RMC is known as well as the start, and it falls between these extremes. The end time is known if next known RMC (say, due to a mechanism change) after the subject time is also stable and shares the same Site, Drive, and Pose indices. If the next known RMC is dynamic, or contains a Site, Drive, or Pose increment, then the end time is ambiguous and the query must be considered mutable.

4.5. *Unsolicited Motion*

Dynamic RMC's cover the case where the rover is known to be in motion. However, the rover can also move when it's not "supposed" to. This is called unsolicited motion, and it can only be detected via ground analysis.

There are two primary types of unsolicited motion: due to mechanism motion, and due to external events. Although similar, they differ in some details.

4.5.1. Mechanism-Induced

This occurs because some mechanism moved, such as the arm pushing on a rock, without the Pose value being updated. In this case, the mechanism RMC values change, but the rover's attitude knowledge does not.

Because the mechanism portions of the RMC were incremented, the RMC can be used as a clock to indicate when the unsolicited motion occurred. This is preferred to using time, as it works with stable RMC's and the new pose can be more easily used in subsequent RMC's.

Entry of a mechanism-induced unsolicited move requires a full 10-element RMC, and the value applies to all subsequent RMC's until the next RMC entry is found for the view.

4.5.2. External Events

This occurs when something external happens, such as the wind pushing the rover off a rock on which it was precariously balanced. In this case, the entire RMC is identical before and after the event. The only way to specify exactly when the move occurred is to use the SCLK, similar to dynamic RMC's.

The ambiguity extends only to the specific RMC value in which the change occurred. For any RMC's before or after that one, the external event looks just like a mechanism-induced unsolicited move. Time only matters within the single RMC. Lest one think this is not relevant, consider the case where the rover is sleeping overnight when the wind gusts. That single RMC could span many, many hours of time.

Of course it's possible (likely, even) that the specific time the event occurred is unknown. In that case, the external event can be treated just like a mechanism-induced unsolicited move, using an RMC value only with no SCLK.

4.6. *Access to Raw Data*

The primary query to PLACES is to get the rover location at a given time, as described in Section 4.1. This may involve considerable computation, depending on the query. There also exists a complete set of queries that obtain the raw data from the database. For example, return the contents of a solution, ask what solutions are available for a given frame, ask what solutions are in a view, etc. The complete set is described in Section 9.

These queries in principle allow one to get all of the data used to answer the primary query (or any other queries), including re-implementing the coordinate transform algorithm. However, this is likely to be extremely inefficient, because PLACES has access to stored procedures and other database efficiency measures that a user of the raw data does not.

4.7. *SCLK Lookup*

Since PLACES already stores all RMC and SCLK values downlinked for the entire mission, a set of queries exist to find the nearest RMC values (before and after) for a given SCLK, and to find the SCLK range during which an RMC is valid. This information may be useful in contexts outside of rover localization. Note that not every RMC is downlinked with an SCLK, so these queries return

the best available information at the time of the query. There are no “solutions” or views since there is no ambiguity in the RMC-SCLK mapping. Nevertheless, the values should not be considered immutable unless an exact (or unambiguous, see Section 4.4.1) match is found; new RMC-SCLK mappings could be entered at any time. Therefore the results also include the immutable flag.

4.8. *Saved Frame Lookup*

The Saved Frame (see Section 2.4) is simply an alias used by the rover for a frame that is saved onboard. It plays no part in PLACES; the corresponding RMC value should be used instead. As a convenience, a separate table of Saved Frame Index to RMC mappings is stored in the PLACES database, and can be queried in either direction.

4.9. *Summary of Immutability*

Since immutability of results is a top-level requirement for PLACES, this section describes the situations in which the results are *not* immutable. All results contain a flag indicating their immutability status.

Immutability is lost when any of these conditions apply:

- The Open portion of a view is referenced in any way.
- A time-based query is used, which translates to a dynamic RMC where the exact time is not available in the database.
- A time-based query is used, which cannot be unambiguously assigned to a stable RMC (see Section 4.4.1).
- An SCLK lookup, in either direction, does not provide an exact or unambiguous match.
- A caret (^, meaning last) is used in the query (e.g. (Site, Drive, ^), for RMC or in a view name.
- An at sign (@, meaning first) is used in the query, when it does not resolve to 0.

The use of ^ or @ means the query is mutable because there could always be higher (or lower, if @ is non-0) values added later. However, if the ^ or @ are resolved to the actual number that was used by the query, the query could become immutable. The resolved values (e.g. actual view name, actual RMC) are included in the returned XML. To indicate this situation, an additional flag may be present if immutable=false. If the *only* reason the query is not immutable is because of the use of ^ or @, so a resolved version of the same query would be immutable, then an additional flag, immutable_if_resolved=true is included in the results. This flag does not appear unless the value is true (i.e. you will never see immutable_if_resolved=false).

4.10. *XML Summary Files and PDS Archive*

PLACES is an active web service. This requires a network connection to use, and communication over the network. Some applications cannot support these requirements, and thus cannot use

PLACES directly. In addition, the PDS archive is not set up to handle dynamic Web services, and the database cannot be guaranteed to remain active after the Project terminates.

In order to accommodate these applications, data for certain views will be periodically extracted and saved in Summary XML files. These can be distributed for use by the non-network applications or for archive.

The specific contents of these files, as well as the operational decisions as to which views to extract, are **TBD**. It is unlikely that such files will contain dynamic RMC's due to space reasons; the files would become quite unwieldy and dynamic RMC queries should be rare. The files however should answer the primary query, in the form "what is the position or orientation of the rover at all stable RMC's within a certain range, according to a specific view". The reference frames for each RMC would likely be the enclosing Site.

The PDS archive will also contain a dump of all raw data as a deep archive (most likely in an XML format related to the query results). However, the XML summary files are more likely to be of use to future researchers, because they require no computation to use, and thus should be considered the primary PDS archive.

5. Creating New Localizations

This section describes how to get new solutions into PLACES, and how to create and use views.

5.1. *Steps Required*

In order to put a new localization into PLACES, the following steps need to be followed:

- Perform the localization
- Create a SolutionID, if needed
- Create a View, if needed
- Create a Solution
- Add the Solution to the View

Note that RMC resources are automatically created when needed

5.2. *Metadata*

Most resources have associated with them a set of metadata. This metadata includes things like who created the resource, why it was created, a description of it, who can update it, etc. Use of this metadata is not mandatory but it is highly recommended for documentation. For example, a solution ID might be created with user="rgd", institution="JPL-MIPL", method="marsnav image analysis", and description="Compensate for wheel slippage during rover bump in the middle of the BigCrater panorama". Such metadata helps users determine which solutions or views they want to use.

Metadata is mutable; it can be changed any time (by authorized users). This enables e.g. the description to be updated as a view is added to. However, changing metadata is generally discouraged.

Metadata is also nullable; it need not be supplied at the time the resource is created.

Note that the numeric position or orientation associated with a solution is considered data, not metadata, and is not nullable - it must be provided when the solution is created.

5.3. *Solution ID's*

Each solution needs to be identified with a solution ID. Solution ID's can be reused in different frames, but must be unique within one frame and type of solution (position or orientation).

The purpose of having a separate solution ID resource is to enable the metadata (creator, purpose, permissions, etc.) to be stored once rather than with each of possibly hundreds of solutions using the ID.

Note that the solution ID names "telemetry" and "pose_p", where p is any integer, are reserved for system use to store telemetry and cannot be created or used in user-generated solutions.

5.4. RMC's

RMC resources are automatically created when needed. However, they can also be manually created. RMC resources may exist for no other reason than to provide richer RMC-SCLK mapping; i.e. they do not have to have any localization solutions attached to them.

Although it is most natural to think of RMC's in terms of Rover frames, the RMC resource is used to hold the instance counter for Site and Orbital frames as well, in which case only the first index is used.

5.5. Solutions

The Solution resource joins the Solution ID with the RMC, and contains the numerical values (position or orientation) for the localization. Position and Orientation solutions are distinct, and the Solution ID and RMC combination may be repeated for each type.

The Solution contains three additional critical pieces of information: the Reference frame, the Derived-from frame (described in the next section), and the View used for both.

The Reference frame and View specify the coordinate frame in which the numerical values are expressed. So for a position solution, the position values are the offset from the Reference frame to the solution's frame. For an orientation solution, the values are a quaternion that represents the rotation from the Reference frame to the solution's frame. Every numerical offset or rotation value must have a Reference, or the numbers are meaningless, and must have a View to be able to put the numbers into a global context.

Note that when creating a solution, it is legal to use the caret (^) syntax for both the RMC and solution IDs to indicate the highest available number. Since this is resolved at the time the solution is created to a specific RMC or solution ID, doing so does not harm the immutability of the solution.

5.6. Using the Derived-from Frame

This section talks about positions/offsets for convenience; it applies equally well to orientations/rotations.

Localization solutions have to be derived from something. They specify a new offset or rotation relative to some other frame. This frame need not be the same as the Reference frame! In other words, the source of the localization (the Derived-from frame) does not have to match the way in which the numbers are expressed (the Reference frame). See Figure 1.

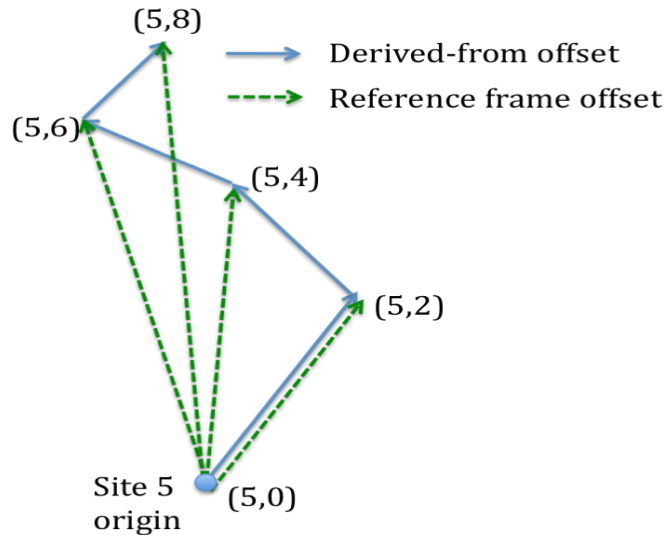


Figure 1: Derived-from and Reference frames for a drive sequence

The archetypal example of this is the case of the rover counting wheel rotations onboard. As the rover drives, it counts wheel rotations and integrates them in order to determine how far it went. All the rover can really determine is how far the *last* drive arc was. But, the position is *expressed* in the Site frame. Consider the example in Figure 2. Let's say that for site 5, drive 12 is located at 5.0 meters (consider only the X coordinate for clarity). The distance from drive 12 to 14 was 2.0 meters, and from drive 14 to 16 was 1.0 meters. That means frame (5,14) has a derived-from of (5,12), and a location of 7.0 meters relative to Site 5. Frame (5,16) is expressed as 8.0 meters from Site 5, even though it's derived-from frame is (5,14). Now say it is determined later that the drive from 12 to 14 was shorter than expected due to wheel slippage, for a distance of only 1.5 meters instead of 2.0. This is shown on the right side of Figure 2. The location as measured in the Site 5 frame for (5,14) is now 6.5. But the drive from 14 to 16 is still valid, at 1.0 meters. That means the position of (5,16) as measured in Site 4 should now be 7.5 meters instead of 8.0. The relationship between frame (5,16) and its derived-from frame (5,14) did not change - it's still 1.0 meters. But the relationship between (5,16) and its Reference frame (Site 5) did change.

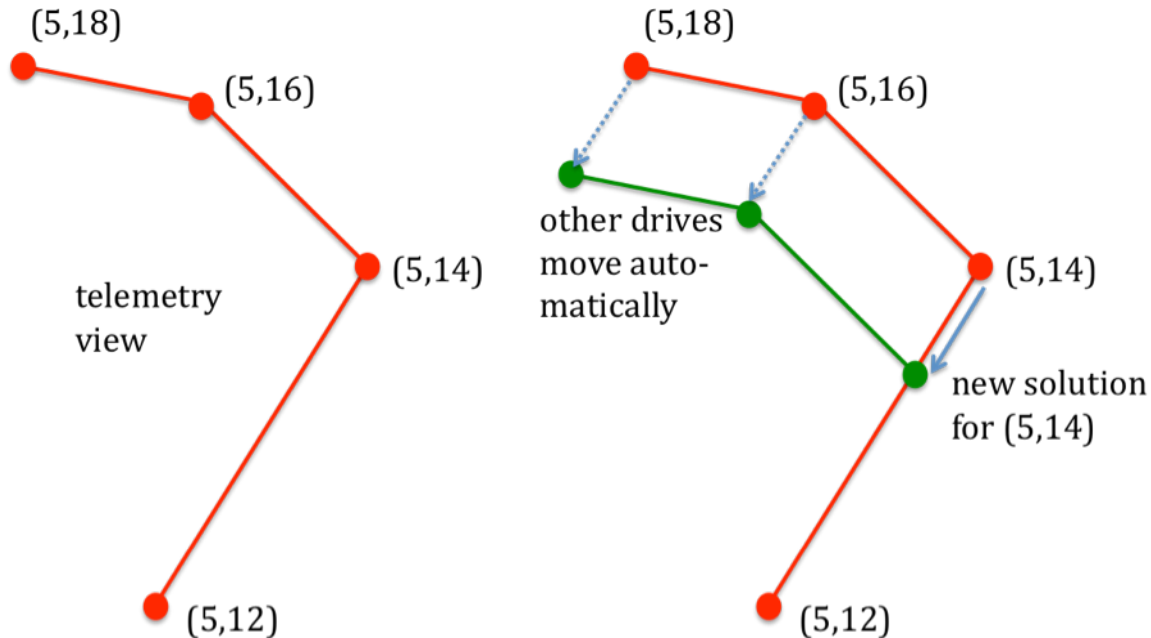


Figure 2: Result of moving one solution in a chain of Derived-from frames

This is the key point: for a given solution, the numeric relationship between the frame and its Derived-from frame does not change - even if the numbers are expressed in a different Reference frame. If the Derived-from frame moves in a different view, the solution frame moves along with it. PLACES keeps track of all this automatically, computing and storing the offset from the Derived-from frame (see Section 6.2.1).

Thus, when creating a new solution, it is important to get both the Reference and Derived-From frames right. Reference is easy; it's the frame in which the numbers are expressed. For all telemetry solutions, that's a Site frame. Most ground solutions will either be expressed in a Site frame or an Orbital frame.

5.6.1. Rules of Thumb

But what should the Derived-from be? Here are some rules of thumb that might help.

In almost all cases, Derived-from will be the prior Drive, the Site, or an Orbital frame.

For “telemetry” solutions, Drive 0 for any site uses the Site frame for its Derived-from. Any other even Drive uses Drive $n-2$ as its Derived-from in “telemetry”. This establishes the proper chain as shown in Figure 1 and Figure 2. For rules regarding dynamic RMC's, see Section 5.6.5.

If the solution is the result of comparing images from the current location and a previous location, then the Derived-from frame is the previous location (where the “before” images were taken). This applies to onboard visodom as well as ground image comparisons.

If the solution is the result of finding the rover on an orbital map, then Derived-from is the orbital map.

If the solution is the result of a bundle adjustment, then the solution is really the result of combining data from many different places. In this case, the bundle adjustment has already obtained an optimal solution; you are unlikely to want the solution to move if a segment in the middle is adjusted. Therefore the derived-from frame should be the same for all solutions participating in the adjustment, and should be some frame preceding all of them. Root (the landing site) is a good candidate, as is an orbital frame.

If you cannot logically determine what the Derived-from frame should be, make it the same as the Reference frame.

Note that both the Derived-from and Reference frames must be earlier in time than (or equal to) the solution's frame (orbital frames are considered to have times fractionally later than Root for this purpose, in the order of the orbital frame ID). This prevents cycles when resolving their values. Although Reference frames in the database must honor this time ordering, queries may request that a frame be expressed relative to a later reference frame.

It is far more efficient if the Derived-from and Reference frames are the same for a given solution. But do not let efficiency considerations get in the way of properly expressing these frames.

Orbital frames should have their Derived-from and Reference frames the same, generally Root.

Because the typical usage is for the Reference to be a Site frame, if the Derived-from is some position within the site, it should generally be expressed as Local Level rather than Rover. That allows the position manipulations described above to occur without having to worry about rotations between different Rover frames, because by definition the Site and all of its Local Levels are parallel.

5.6.2. Derived-from and Sites

When a Site frame is declared, the position of that site frame with respect to the previous frame is saved and downlinked as a site offset. These site offsets provide a link between each site and its predecessor.

But how is that site offset derived? It's derived as the position of the rover at the time the Site frame is declared. So in reality, the Derived-from of a Site frame should be the last Rover frame before the Site was declared.

However, that is impractical for two reasons. First, the ultimate Rover frame at the time the site is declared may not (ever) be known. In the case of rover drives, we know the Derived-from is drive $n-2$; if $n-2$ is not present in the database, we can use a prior drive with no loss of accuracy (see Section 6.4). However, in the case of a new site, we may not know the proper drive index to use - and picking the wrong one would lead to incorrect results.

A second factor is efficiency - it is not practical to check every single drive throughout the entire mission when doing coordinate transforms. Once the derived-from chain gets to a site frame, site-to-site transforms can be used instead.

For these reasons, the Derived-from for a Site frame for “telemetry” is set to the previous Site frame (just as its Reference frame is).

The implication of this is that localizations within a site will not affect anything in the next site without explicit user action. This actually matches typical user expectations better, since sites are often considered to be independent realms. If you want the next site to be affected, create a solution for the next site containing the combined results of the intra-site adjustments.

Another option would be to create a new Site solution with a Derived-from of the last drive from the previous site - i.e. create the “true” situation and let PLACES figure it out. This is acceptable as long as the user knows the proper drive to use and can accept the performance penalty. But this is a user choice - not something PLACES does by default.

5.6.3. Derived-from and Views

There is one other important factor to note regarding Derived-from and Reference frames: they must use the same View. This view is attached to the Reference frame by convention, but it applies equally to both. As described above, solutions are ultimately described by the offset from the Derived-from to the solution frame. This offset is computed, if possible, when the solution is submitted. Computing this offset (the full algorithm is described in Section 6.2.1) requires determining the position of the Derived-from frame in the Reference frame using the given view. Once this offset is computed, the reference view is no longer needed.

Note that this is an easy requirement to meet; if you do not have a single view covering both the Reference and Derived-from frames, simply create a child of the Reference view and add the relevant Derived-from solutions to it.

But what if the Derived-from frame does not exist? This situation commonly occurs in “telemetry” since not every drive gets downlinked. It is difficult to come up with a use case for this situation during ground analysis; in order to figure out where you are relative to the Derived-from you really need information about the Derived-from frame. Yet it could happen; perhaps an analysis of motor currents suggests a wheel slipped by 10%.

If the Derived-from frame does not exist in the given View at the time the solution is created, then the offset cannot be computed at creation time. Rather, computation is deferred until it is needed. This means that, at the time of solution submission, the view must be closed up to both the Reference and Derived-from frames in order to maintain consistency. Otherwise the values could change between solution submission and use, creating inconsistencies. Since this normally occurs only with telemetry, which is permanently closed, this should not be an issue. An error results and the solution is rejected if these criteria are not met.

Note that use of an open view is fine if Derived-from exists because the database computes the offset at submission time. So the offset reflects the state of the view at the time of submission. If the open view changes later, the solution offset does not change.

If you want to provide your own offsets and not let PLACES calculate them, simply set the Reference frame to be the same as the Derived-from frame. The given numeric values will become

the offset with no further alteration. When the Reference and Derived-from frames are the same, the supplied view is not relevant and can be anything.

Note that even though the original position relative to the Reference frame is not needed any more once the offset is computed, it is stored in the database for traceability reasons.

5.6.4. Derived-from and Orientations

The above discussion of the Derived-from frame centered around position. Orientations also have a Derived-from frame, which should be used in the same way if the orientation is derived from image analysis.

However, most orientations are determined using the IMU, accelerometers, or a sun-find. These methods of determining orientation provide what might be thought of as absolute orientations but are actually relative to the surface (north/down). For this reason, these solutions should use the Site frame as their derived-from.

5.6.5. Derived-from and Dynamic RMC's

During drives, the rover positions in a dynamic RMC may be updated up to 8 times per second. Technically each 1/8 of a second update has a Derived-from of the preceding 1/8 of a second. However, due the enormous potential number of updates, and the potential for significant gaps in the returned data, maintaining such a fine grain of Derived-from is not tractable.

Combine this with the limited value in modeling the exact drive path, the low probability of localizing frames within a dynamic RMC, and the computational burden and accumulation of roundoff error involved in interpreting such a long chain of Derived-from frames, and it does not make sense to maintain such a chain. Therefore the “telemetry” solutions will set the Derived-from frame for all driving dynamic RMC's to be the immediately prior stable RMC (as a 3-element reference). This is most likely a Local Level frame, but could be a Site frame if the drive is 0.

For orientations, including both drives and arm motions, if the natural Derived-from is the site frame, then that's what should be used. Otherwise, the above logic applies. For “telemetry”, it is not always practical to distinguish between IMU/accelerometers/sun find, for which the natural Derived-from is the Site, and visodom results, which should use the previous stable RMC. Because of this ambiguity, and because visual odometry is rarely used to update orientations, dynamic “telemetry” orientations will always have the Site frame as their Derived-from.

Note that it is possible to create one's own solution ID for a set of dynamic RMC's that mimics “telemetry” with a long chain of Derived-from frames, if that is necessary.

5.7. *Dynamic RMC's and Unsolicited Moves*

When creating a solution for a time-based dynamic RMC, it is important to provide the RMC in addition to the SCLK value. This will help improve the RMC-SCLK mapping as well as remove any ambiguity as to which RMC the solution belongs to.

For mechanism-induced unsolicited moves, or those from external events for which no time is available, providing the RMC only is sufficient. However, external events with a time value should be created using both the RMC and the SCLK.

In summary, solutions cannot be created using SCLK alone. If the RMC is truly unknown, do a SCLK->RMC lookup first.

5.8. Creating Views

Creating views is intended to be a fairly lightweight operation. Each view is defined as the contents of a Parent view, plus modifications to that. They logically contain a solution for every RMC, but physically contain only those solutions that are different from their parent.

There is no specific naming convention for views (or solutions), although it is recommended that the user name be included to help ensure uniqueness. However, the following view names are reserved and may not be used:

- telemetry
- tactical
- best_tactical_nnnn
- pose_*

Views are created open; it is up to the user to explicitly close them to whatever point is desired.

5.8.1. Parent Views

Every view has a parent view, which is used to fill in the sparse RMC list of the child and make available a solution for every RMC. This is true until the root of the view tree is reached.

The root of the View tree is “telemetry”. The “telemetry” view has no parent; it is expected to provide a value for every known RMC. It may in fact have a few holes (such as unsolicited moves) which are filled in by the child views, but sibling views not containing the unsolicited move will simply not see the missing RMC.

Once a view is closed to a certain point (defined by an RMC), no modifications are allowed to it prior to that point. This preserves the immutable property of the database. Note that site entries are considered the same as a rover entry with RMC (site,0) for the purpose of view closing. And orbital images are considered the same as Root for this purpose (thus, they are closed if anything in the view is closed).

Child views can be closed only up to the point their parent is closed. This prevents ambiguous situations where the view looks to be closed but is actually mutable because the parent could change.

The “telemetry” view is defined to be completely closed, despite the system being able to add entries to it. This enables it to act as the root for all views. The rationale for why this works is described in Section 6.4.

If the user wants a view that mimics “telemetry”, in that it’s a dense view that is updated out of order as new data arrives, then they should use a standard view and simply not close it. Such an updating view cannot be considered immutable anyway (except for the special case of “telemetry”), so there’s no reason to close it.

5.8.2. Cloning vs. Deep Copy

There are two ways of copying a view: cloning and deep copy.

A clone is a literal copy of the view - same parent, same included solutions. This is useful if you want a closed version of someone else’s view that is still open, or if you specifically do not want to include future changes from their view into yours. Note that if you simply want to extend their closed view, you should make it a parent rather than cloning it. You can also easily reject certain changes by making it a parent and setting the RMC’s in question to use a previous solution, rather than cloning.

A deep copy of level n makes a new view with the parent of the view n levels up the parent chain, and a copy of all solutions back to that point. Essentially this collapses all solution modifications since the parent n levels back into one view. This could be useful if your parent’s parent wasn’t closed but you needed to close your view. In this case you could do a deep copy of level 2, and then close your view. Deep copy should be used with extreme caution. If the number of levels back is too big, you could end up with a dense copy of everything back to “telemetry”, which would have significant database performance impacts.

For this reason, only administrators can do a deep copy. If a user needs this, contact a PLACES administrator.

A deep copy of level 1 is really the same as a clone. However, clones exist as a separate concept because they should be much more commonly used, and are safer. Cloning of “telemetry” is not allowed for safety reasons, as it would result in a dense copy of everything. If you really need to clone telemetry, use the deep copy mechanism instead.

Note that the default when querying a view is to return just the solutions particular to a given view - i.e. the sparse set, not the ones that come from the parent. A flag on the query allows one to return the parent solutions too.

5.9. Groups and Users

Although anyone who has access to PLACES can query any solution or view, the creator of solution ID’s and views can control who can update them. This prevents someone from inadvertently (or maliciously) changing someone else’s solutions or views.

The mechanism for doing this is the Group. A Group is made up of Users or other Groups. Only users in the Group attached to the View or Solution ID may modify those resources.

If the group is null, anyone can update the resource.

Because group permissions do not affect the actual contents of the solutions, they can be changed at will (by someone who is a member of the old group). In other words, group assignments and the group attached to a resource are mutable at will.

It is anticipated that certain standard groups will be created, such as “tactical_ops”, “mipl”, “science_team”, “chemcam_team”, “UofA”, “MSSS”, etc. The memberships of these groups may change without the group ID itself changing. Groups can also contain other groups; for example a view might use a group that allows “chemcam_team” and “mipl” to modify it but nobody else.

Overall access control and authentication is provided by LDAP (see Section 7).

6. Detailed Design Topics

This section contains details relating to the design of PLACES. This information will be useful to completely understand how PLACES works but may not be needed by general users.

6.1. *Are Sites Parallel?*

The question as to whether site frames are parallel to each other is somewhat controversial within the MSL project. On the one hand, sites are defined as +X pointing north and +Z pointing down according to the local gravity vector. On a curved surface such as Mars, this means site frames are not parallel; they mimic lines of longitude, which are not parallel to each other.

On the other hand, within the range of locations relevant to operations, the difference is negligible. The difference in Z due to planet curvature is on the order of 15cm over a 1km traverse, far less than the measurement error over that distance. Therefore, the rover onboard assumes site frames are parallel.

This is reasonable for operations; it is rare that data from over 1km away are used during operations. However, for precision large-scale mapping, it could make a difference.

PLACES remains agnostic on this issue, letting the user decide. The Site frame stores, like any other frame, both a position and an orientation relative to its reference frame. For the “telemetry” view, the orientation quaternion is set to identity (no rotation relative to the reference), which mimics the onboard software and makes all sites parallel. Users are free, however, to define a site solution which contains a rotation term, in order to compensate for planet curvature.

6.2. *Coordinate Transform Algorithm*

The PLACES primary query often requires that coordinate transforms be done. These transforms can change the Reference frame, the view that the frame is expressed in, or both. The algorithms involved are explained in this section. Note that from a user perspective, this should be transparent; the information is provided here in case detailed knowledge of the algorithm becomes necessary.

Adding or subtracting positions requires rotating one of the frames so the coordinate axes are parallel, if they are not already. This involves performing a similar query to determine the rotation between the two frames, and then applying that quaternion to one of the frames (usually the source frame, so the numbers end up expressed in the destination frame).

As a reminder, this section talks about positions (and offsets) for convenience. Orientations (rotations) use the same algorithm: adding offsets means to composite rotations, while subtracting means to invert the rotation before compositing. Position plays no part in orientation determination. The primary difference with rotations is that the Derived-from frame is often the Site, making Derived-from chains much shorter.

The coordinate transform algorithms are described in a recursive manner, as they are easiest to understand that way. The implementation may convert this to an equivalent iterative algorithm for efficiency.

Both the set of Reference frames and the set of Derived-from frames create tree structures, ultimately rooted at the Root frame. Because both Derived-from and Reference frames for a given solution must be earlier in time, this is a proper tree with no cycles or merges.

The coordinate transform algorithm works by obtaining the combined offset and orientation (rotation) between the source and destination frames (as described below), and then applying those to the given point or unit vector to get the final result.

6.2.1. Determining the Derived-from Offset

As described in Section 5.6, the fundamental definition of a solution is its offset from the Derived-from frame. This offset is a constant; if it were to change you would have a different solution.

Normally, the offset is computed at the time the solution is submitted to PLACES. Under certain conditions, however, the computation is deferred until needed. This is described in detail in Section 5.6.3.

Regardless of when the computation is done, it is done in the same way: find the location of Derived-from in the Reference frame using the Reference view, rotate if necessary to align the axes, then subtract that from the location of the solution in the reference frame/view (supplied with the solution) to obtain the offset. If the Derived-from and Reference frames are the same, the offset is simply the numeric value supplied with that solution.

If the Derived-from frame is not available, say because it was never downlinked, then we simply search backwards to find the most recent available frame before Derived-from, and use that (temporarily, for this query only) as if it were Derived-from, in order to compute the offset. This is valid because of the special nature of “telemetry” (see Section 6.4); if we have no information on intervening drives then they cannot be changed from what’s in “telemetry”, and “telemetry” comprises a self-consistent set of values. And since the view must be closed in this case, only new “telemetry” solutions can appear. Such offsets are temporary; only if the proper Derived-from frame appears is the offset cached for later use.

6.2.2. Calculating a Solution in its Own Reference Frame

In order to calculate the position of a solution as expressed in its own reference frame, using any arbitrary view, these steps are followed:

- If Derived-from equals the Reference frame, return the offset (either the offset or the original location in the Reference frame; both are equal).
- If Derived-from does not equal the Reference frame, compute the location of Derived-from in the Reference frame using the desired view. Note that this is a recursive call to the primary query. Rotate if necessary to align the axes, then add the offset (calculate on-the-fly if needed) and return.

It is interesting to note that the Reference view doesn’t matter in this query. Once the offset is derived, the Reference view no longer matters. Of course if the offset is computed on-the-fly, the Reference view matters for that computation only.

6.2.3. Changing Reference Frames

Changing the reference frame as well as the view represents coordinate transforms in their full generality.

First, get the solution in its own reference frame using the desired view, as described in Section 6.2.2.

Then, get the location of the solution's reference frame as expressed in the desired reference frame (using the desired view). This is a recursive primary query. Rotate the frames as necessary, add these two values, and return.

This works simply, as long as the desired reference frame is on the path from the solution's reference frame to the root. Once some intermediate frame is expressed in the ultimately desired reference frame, the recursion stops.

But what if that's not the case? Say if you want Rover:(5,12) as expressed in Site 6? In that case, the algorithm will work itself all the way to the root node of the tree without finding an answer. So then, a similar process happens on the other side: the position of the desired reference is computed, all the way back to the root if necessary. Once both references (the solution's and the desired) are expressed in the root node (all using the desired view), subtracting them gives the offset between them, which ends the recursion.

In reality, this need not go all the way to root; once a common node is found in the trees of both reference frames, the process can stop at that point. The frames can then be related to each other, which allows the final answer to be computed.

6.3. *Frames and Subframes*

As noted in Section 4.3, stable RMC's are stored using only their (site,drive) components, potentially with a "pose_p" solution ID. The mechanism RMC components are not needed, nor are they stored. It is expected that the bulk of PLACES usage will be using these stable RMC's.

On the other hand, dynamic RMC's and unsolicited moves are stored using the full 10-element RMC.

As an implementation convenience, these types of frames are stored separately in the database. The dynamic RMC's are "subframes", as it were, of the "primary" stable RMC's.

This distinction makes very little difference to the user. However, when returning metadata about a 2-element RMC, there is a "subframe" field which indicates whether or not any of these subframes (dynamic or unsolicited move) exist for this RMC. A list of these subframes can then be obtained. Note that this is associated with the RMC itself, not with any particular View, so even if there is a subframe for the given RMC, it may or may not be relevant in the View you are working with.

Nevertheless, this facility can be handy to see if there are any unsolicited moves attached to a stable RMC.

Primary frames and subframes act identically to all queries of any kind, except where noted in this document.

6.4. Why “telemetry” is Special

It has been noted numerous times in this document that the “telemetry” view is special - it is a permanently closed view, yet new RMC values can be added to it. This section explains why that is the case.

Onboard the rover, “telemetry” represents a complete self-consistent set of positions. Every RMC was known while the rover was at that RMC, and offsets to the next RMC were also known. Although the rover doesn’t store most of these values once it moves on (some are stored via the “saved frame” mechanism), they were all available at the time the rover was there.

Thus, every value in “telemetry” was computed incorporating full knowledge of every other value. There are no holes in this knowledge onboard, and it is a complete self-consistent set of data.

However, not everything gets downlinked to the ground. This creates holes in our *knowledge* of this complete telemetry RMC database. It is only our knowledge that has holes, however - not the computations involved in creating the values.

Therefore, when we fill in a hole later, say due to a delayed transmission, this new information may fill in gaps - but it cannot in any way modify anything else that is known about the “telemetry” solution, because everything else already took the “hole”’s information into account when it was originally computed.

The practical upshot of this is that filling in holes for “telemetry” is not the same as adding a new solution - it does not change anything else. Therefore it is safe to add these values to the otherwise closed “telemetry” view.

Adding any other kind of solution by definition involves a change to some rover positions, relative to telemetry if nothing else. This makes adding any other solutions to closed views unsafe - such solutions seek to change the state of the solution tree and therefore violate the immutability constraint.

6.5. Mapping of Orbital Images

Orbital images are referenced using a single index. Each orbital image stands on its own, defining a coordinate system relative to some parent (often Root but possibly other orbital images).

Orbital images must be in a rectangular projection, with north up in the image. The line (Y) dimension of the image is aligned north/south with the sample (X) dimension aligned east/west. There is a constant pixel scale (meters/pixel) in both directions. The X and Y scales need not be the same, but it is recommended they be.

Data associated with an orbital image includes:

- Image identification (filename or URL)

- Pixel scale
- Size in pixels
- Location on the image (in pixel space) of the XY(Z) origin
- Standard descriptive metadata information (originator, description, etc.) as with any other solution ID or view.

This data, except for the descriptive metadata, is immutable once set.

Additionally, like any other frame, orbital images contain:

- Reference frame identification (usually Root)
- Transform from XYZ origin to the reference frame origin
- Derived-from frame (should be the same as reference frame)

The transform can include an offset and/or orientation just like any other frame reference. This allows multiple maps to all have North be up yet have a rotation between them to cover large distances. This rotation should be identity (no rotation) in most cases, although rotation between Orbital frames is supported.

Note that the origin of the XYZ frame in the image need not coincide with the origin of Root (the landing site). This allows the XYZ coordinates of the orbital image to be put in a convenient location (e.g. center or corner of image) independently of the where Root is determined to be. This may be backwards from intuition: the landing site itself is likely to be localized relative to an orbital image, but since Root is the ultimate reference, the orbital image is actually moved (via its offset) to where the landing site is.

Orbital images do not include a Z component. Support for DEM's could be added in the future. However, locations relative to an orbital image must include a Z. It is recommended the original Z from telemetry be used, unless you have some method to determine something better.

Orbital images are ordered by their index. If one uses another as its Reference frame, the Reference must have a lower index number. This prevents cycles in the tree. All orbital images are considered to be after Root but before any other RMC value.

Although nothing enforces or requires this, orbital images should have the same Reference and Derived-from frames.

6.6. Back Doors

As has been stated numerous times, PLACES is for the most part an immutable database - once something has been written, it cannot be changed. This is part of the contract; users can rely on queries always returning the same values, if they are marked as immutable.

However, a robust operational system cannot have such rigid rules. There must be a way to change things in case bad data gets entered or the database otherwise gets corrupted. Operators must be able to react to extraordinary situations.

Therefore there are some “back door” mechanisms that allow modification or deletion of solutions, modification of closed views, reopening of closed views, etc. These back doors require special privileges, and will only be used with the concurrence of all affected parties.

6.7. *Sources of Telemetry Data*

The “telemetry” view will be filled in using as many data sources as possible. Any telemetry channel that provides position or orientation value for a given RMC is a potential source. The specific list of sources will likely expand over time, but should include:

- Image data (using the IDPH or labels)
- Non-image instrument data (using the common telemetry product header)
- Mobility and Arm data products
- EVR's

This is a representative list; specific sources (other than images) are still **TBD**.

6.8. *Approvals*

Earlier versions of this user guide discussed Approvals. The idea was to enable someone with the authority to do so, to mark certain solutions as “approved” for a specific use, such as being included in “best_tactical_nnnn”. At the point the view was ready to be created, it would gather everything marked with the “best_tactical” approval and add it to the view.

However, this mechanism can be completely implemented using open views. The approver can simply enter solutions into a specific open view. The solutions in this view can then be copied out and added to the view under construction.

7. PLACES and MER

This document so far has talked about PLACES in the context of MSL. But PLACES can also be used with MER data, both as a testbed for MSL and as a useful tool in its own right.

Although the MER and MSL versions of PLACES will use separate physical databases running on separate servers (at least separate virtual servers), the database schema and supporting code and queries are similar between them. There are some differences however in how the database should be used for MER. This section discusses those differences.

The intent is that querying for data should look the same as MSL, as much as possible. Differences mostly occur when uploading telemetry data.

Because there are two MER rovers, each one will have its own database and server. In other words, the base URL will be different; the queries will be the same between them.

7.1. *RMC Differences*

The most obvious difference with MER is that the RMC is only 5 elements long: Site, Drive, IDD, PMA, HGA. Although some of the names are different, each element has a direct analogue in MSL:

- Site -> Site
- Drive -> Drive
- IDD -> Arm
- PMA -> Mast
- HGA -> HGA

The user interface will thus be slightly different for MER. The XML tags (input and output) for RMC elements will use the MER names. The query syntax, instead of the full 10 elements, will use just 5 elements (the simulated Pose, see Section 7.6, does not need to be part of the RMC “clock”). The 2 and 3 element queries are the same.

Note that for MER, the first site (landing site) is Site 0, not Site 1.

MER has no concept of “saved frame”, so the saved frame lookup table is not needed.

7.2. *Lack of Pose Index*

The most profound difference between the two missions is that MER lacks a Pose index. This makes it very difficult to distinguish between the stable, dynamic, and unsolicited move cases on MER.

The result of this is that the orientation can change at any RMC, due to a sun-find, or reading of the IMU or accelerometers. However, these actions are really Pose updates - they update pose knowledge, but do not reflect a physical change in the rover. Seven years of operational

experience has shown that arm-induced movement is very rare and very small when it does occur, and true unsolicited motion has not been observed.

For that reason, any changes to the telemetered orientation between drives are treated like a Pose update. If a true unsolicited or mechanism-induced motion occurs, that information can be entered in additional solutions.

7.3. *Queries*

Because the Pose index is not special for full 10-element (5-element for MER) or SCLK-based queries, these queries act identically on MER or MSL. Asking for the full RMC (or using the SCLK) will give you whichever Pose update was active at that specific time. In other words, using this query mechanism will make it look like the Pose updates were unsolicited or mechanism-induced motions.

For this reason, the 2-element (site, drive) or 3-element (site, drive, pose) queries are greatly preferred, even more so for MER than MSL. The Pose index is simulated (see Section 7.6) which makes the database look, to the user, like a simple set of (site, drive) entries with potentially multiple Pose solutions per entry.

7.4. *Immutability*

Because of the lack of the Pose index, it is harder to determine what results are immutable. Therefore, more queries will come back saying they are mutable, simply because we cannot tell if there are other updates. We err on the side of caution, only marking something immutable if it truly is.

7.5. *Dynamic Drives*

MER does not support the even/odd concept for the Drive RMC index. Therefore there are no dynamic drive situations; we assume that any Drive RMC (with mechanism indices all 0) is stable. MER rarely does much other activity while actually driving, so this is not a big limitation.

7.6. *Simulating Pose*

In order to simulate Pose, we make an assumption: if the position and orientation are exactly the same, they represent the same Pose. If the Pose changes, it will not return to exactly the same value in the same Drive. This assumption allows us to group RMC's together by having identical pose.

The Pose "index" for each of these groups will be some SCLK within the group. Which SCLK is used should be treated as undefined (it's actually the first one seen with that unique pose value, which may not be the first in the group). Using SCLK allows ordering of poses to be maintained even if data comes down out of order.

So like the MSL "pose_*p*" solution, there will be a "pose_*p*" solution for each pose group, where *p* is an arbitrary SCLK within the group. This allows the "pose_^" query to be used to get the last available pose, or "pose_@" to get the first. The (site,drive,pose) query syntax can also be used

here, although the SCLK must match exactly. The first and last symbols will be far more generally useful in this query.

MER will have an extra database table that says, for each pose group, what are the minimum and maximum RMC's and SCLK's that have been seen for this pose group. Thus if you need to find a specific pose, this table can be queried to get the pose name (or if you only need the numbers, a full 5-element or SCLK query can be used).

The limits in this table are mutable, in that they can expand outward as new data is found that should be a member of the group. The limits cannot shrink, however, so once an RMC is identified as being a member of a group, it will always be a member of the same group.

This table will be used to determine immutability for telemetry solutions, for full queries. If a given RMC is in between the limits of a group, then it is considered stable and immutable. If however it is between groups, then there is no way to know which group (before or after) it should be attached to, or if there are new groups that might appear. In this case, the RMC is considered to belong to the latest group prior to the RMC, but is mutable.

One problem with this system is that, unless we see data for the first RMC in a drive (i.e. all mechanism indices equal 0), then we cannot unambiguously assign a value for "telemetry" for that drive. There's always a chance that the first RMC could have a different value. Therefore querying "telemetry" using the (site,drive) mechanism will often return a "no solution" status. For this reason, "tactical" should be used in preference to "telemetry" - even more so than on MSL. The "tactical" view will have an entry for every (site,drive) where we have seen any data, because it will use the first available pose at the time the "tactical" view is updated.

8. How to Access PLACES

This section describes how to access PLACES.

8.1. *ReST Web Service*

PLACES is accessed via a ReST style Web service. ReST stands for Representational State Transfer, which is a meaningless acronym for a very useful concept. ReST systems consist of resources, which are named and accessed via URL's. These URLs are used in HTTP Get and Put operations, using XML as the return data for Get's, and as the input payload for Put's. Transactions are idempotent, meaning the same transaction can be submitted multiple times without harm. For example, a request to create a View can be submitted multiple times. The first will succeed; the others will get a status return indicating the view has already been created.

Another important property of a ReST interface is that of stateless queries: a query stands alone. This does not necessarily mean that the query will always return the same answer; the underlying database may have changed. But the query means the same thing; there is no context or environment or setup required for a query (except for security logins).

8.2. *Authorization and Authentication*

Security in PLACES is managed via the JPL LDAP server. Users must log in with their JPL username/password and must be in an LDAP group that allows access to PLACES. Contact the PLACES administrators for help with this.

Automated processes cannot use an interactive username/password. In order to support these, there is a token authentication mechanism available for "password-less" access. Contact the PLACES administrators or developers for further information.

8.3. *PLACES Resources*

In the ReST-style web service paradigm, objects that can be accessed using the web service are called "resources". Each resource has a URL address (see Section 9), and can be queried or updated in different ways. Each resource has an XML representation as well, which is returned in query results. The resources in PLACES are:

- RMC
- SolutionID
- Solution
- View
- User
- Group

8.4. *Get and Put Operations*

Data access in PLACES is via HTTP Get operations. All inputs are included in the URL. These Get operations can be performed in any web browser, by simply typing the URL in the address window. The information is returned in XML format.

For ease of access, XSL stylesheets convert the XML to HTML for display in browsers, when accessed via a browser. This makes the returned data easier to read. Most browsers have a “view source” option which allows viewing the raw XML.

Updates happen via HTTP Put operations. Since most updates require more information than can comfortably fit in a URL, they require an XML payload file, containing the information to update. The Put operations return an HTTP status code, which tell if the operation succeeded or not. Put operations are harder to do in most browsers, but there are plugins for some, such as “poster” for Firefox, which allow access to Put operations and the payload.

In addition, many of the common Put operations will be implemented via HTML forms for ease of use. Combined with the stylesheets, this makes most of PLACES available for interactive, browser-based use.

All modern programming languages have methods for accessing URL's. For Unix/Linux command-line or script access, the “curl” program allows access to both Get and Put operations.

8.5. *Base URLs*

There are several instances of the PLACES server. For example, there is an Operations server, a Test server, and a Development (sandbox) server for each mission (MSL and MER). The Development sandbox servers are free-for-all zones in which people can create anything they want for use in testing their software. It can be cleared or updated at any time. The Test servers are used for formal testing, while the Ops servers contain the real operational data.

The server to use is specified via a base URL, which might look something like “http://places-msldev.jpl.nasa.gov/msldev/”. The base should be applied to all the URL's shown in sections 9 and 0, where it is indicated via “\$HOST”.

The base URLs for the various servers are operational decisions not documented here. To find the servers, visit the PLACES home page, which is at:

`https://jplwiki.jpl.nasa.gov:8443/display/places/Home`

TBD: the page may change to `http://places.jpl.nasa.gov` in the future.

9. URLs and Query Results

PLACES is accessed via ReST resources. This essentially means that all the resources in places can be accessed through URLs in a browser. There are two main types of operations that can happen: Gets and Puts. A “Get” is similar to a read from a database. It’s a way of getting information that is stored in PLACES. To write data into PLACES, a “Put” operation is used. Below we have separated Get and Put queries.

9.1. Query Element Syntax

The URL’s specified in this section contain common elements that are shown in ALL CAPITALS. The syntax of these elements is shown below.

RMC - RMCs take the form of *FRAME(indices)_SCLK*. For example *site(1)*, *rover(3,16)*, *local_level(3,16)_3365567442.16*, *rover_3458297221.1225*, *site_3901268593*, or *rover_subframe(3,4,1,3,3,6,7,6,2,0)_302.33*. And the site, drive, and pose indices can use the ‘^’ notation to find the highest available number.

SCLK – the actual s-clock known to the rover. Examples include 3567866331 and 3355125678.085 .

SOLN_ID – String name of a solution ID. Examples can be “telemetry” or “tactical.” The two wildcards can be used whenever you see “SOLN_ID” as well. So “pose_^” and “rgd_best_@” would give you the highest numbered “pose” and lowest numbered “rgd_best” solutions respectively.

VIEW – Name of a view to look up, like “tactical”. Like “SOLN_ID” names, the ‘^’ and ‘@’ symbols can be used to find the highest and lowest numbered views.

USER –JPL LDAP username.

GROUP – Group names. Can be created by anyone, and can be used to limit write access to solution and view IDs.

PATTERN – Wherever ‘PATTERN’ appears, a free text search will be conducted on whatever is entered (would search for *PATTERN*). For example, a search for ‘tactical’ would find results ‘best_tactical’ and ‘my_tactical’. The ‘*’ wild card can be used as a place holder. So “*_1” would find “pose_1” and “tactical_1.”

FRAME – Simple integer representation of a saved frame.

RETURN – Only used for the primary and translation queries. Can be ‘position,’ ‘orientation,’ or ‘both.’ By default, if not specified, ‘position’ is used.

TYPE - Can be ‘position,’ ‘orientation,’ or ‘both.’ By default, if not specified, ‘both’ is used. Used when adding solutions to a view.

POINT – a point for translation purposes. Is of the form (x,y,z) (with or without parentheses)

DIRECTION – a direction (unit vector) for translation purposes. Is of the form (u,v,w) (with or without parentheses).

SITE,DRIVE,POSE – Used only for the savedFrame resources. All integers. Saved frames only represent these three indices.

9.2. GET Queries

For Get queries, there are several parameters used below to refer to PLACES resources.

Translations

Return Type	URL	Description
Translation	\$HOST/query/primary/VIEW?from=RMC&to=RMC&return=RETURN	The primary query. 'from' and 'to' are required. If no 'return' is specified, only position information is calculated and returned.
Translation	\$HOST/query/translation/point/POINT/VIEW?from=RMC&to=RMC&toView=VIEW	Translation query for a point. 'from,' 'to' and 'toView' are required.
Translation	\$HOST/query/translation/direction/DIRECTION/VIEW?from=RMC&to=RMC&toView=VIEW	Translation query for a point. 'from,' 'to' and 'toView' are required.

RMCs

Return Type	URL	Description
List Of RMCs	\$HOST/rmc/near/SCLK	Returns a list of RMCs. Will return one RMC if the provided SCLK matches exactly, otherwise returns closest matches both greater and less than provided SCLK.
RMC	\$HOST/rmc/RMC	Display the information for a given RMC
List of Solution	\$HOST/rmc/RMC/solution_ids	List all solution IDs for a given

Ids		RMC
List of RMCs	\$HOST/rmcs?from=RMC&to=RMC	List all RMCs within the range
List of Solution IDs	\$HOST/rmcs/solution_ids?from=RMC&to=RMC	Return all solutions IDs that apply to the given RMC range
RMC Metadata	\$HOST/rmc/RMC/metadata	Return a list of key/value pairs for RMC frames. Mostly used for Orbital frames, but can be used for any RMC metadata.

Solutions, Solution IDs

Return Type	Url	Description
List of Solution IDs	\$HOST/solution_ids	List of all solution IDs in the system
List of Solution IDs	\$HOST/solution_ids/PATTERN	All solution IDs matching [PATTERN]
Solution ID Metadata	\$HOST/solution_id/SOLN_ID	Metadata for the solution named [SOLN_ID]
List of Views	\$HOST/solution_id/SOLN_ID/views	List of all views using the solution [SOLN_ID]
List of RMCs	\$HOST/solution_id/SOLN_ID/rmcs?from=RMC&to=RMC	RMCs using solution ID [SOLN_ID] in the RMC range given
Solution Data	\$HOST/solution/SOLN_ID/RMC	Solution (orientation, position) data for [SOLN_ID] at [RMC]

Views

Return Type	URL	Description
List of All Views	\$HOST/views	List all views in the system
List of All Views	\$HOST/views/PATTERN	List all views matching a certain pattern
View Metadata	\$HOST/view/VIEW	Get metadata for view named [VIEW]
Solution for View/RMC	\$HOST/view/VIEW/rmc/RMC	Get the solution in view [VIEW] at the RMC listed
List of RMCs	\$HOST/view/VIEW/rmcs?from=RMC&to=RMC	List the RMCs in the view within the specified RMC range

Saved Frames

Return Type	Url	Description
3 element RMC	\$HOST/savedFrame/FRAME	Return the 3 element RMC saved for frame number [FRAME#]
Frame Number	\$HOST/savedFrame/(SITE,DRIVE,POSE)	Return the frame# for a given (RMC)

Users

Return Type	Url	Description
List of Users	\$HOST/user/users	List all users in the system

Groups

Return Type	Url	Description
-------------	-----	-------------

List of Groups	\$HOST/groups	List all groups in the system
List of Users	\$HOST/group/GROUP	List members of the group

Other

Return Type	Url	Description
LDAP information	\$HOST/ldap/GROUP	Gets members for the ldap group [GROUP]
Form	\$HOST/getSecretKey	Form for generating secure token

9.3. Get Query Response Types

Translation

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/translate.xsl"?>
<translate immutable="true" xmlns="http://places.mars.mipl.jpl/xml/schema">
  <view id="telemetry" />
  <reference-rmc href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/SITE(1,0,0)_15.0" site="1" drive="0" pose="0" frame="SITE"/>
  <offset x="4.0" y="3.0" z="2.0"/>
</translate>
```

RMC

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc.xsl"?>
<rmc href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/ROVER_SUBFRAME(1.1.1.1.1.1.1.1.1) 260.0/solution" add_date="2011-09-20 14:59:51.0" actual_sclk="260.0" frame="ROVER_SUBFRAME" site="1" drive="1" pose="1" arm="1" chimra="1" rsm="1" hga="1" drt="1" ic="1" xmlns="http://places.mars.mipl.jpl/xml/schema"/>
```

RMC Metadata

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmcMeta.xsl"?>
```

PLACES User Guide v2.0-B

```
<metadata xmlns="http://places.mars.mipl.jpl/xml/schema">
<item>
<key>image name</key>
<value>orbital_image_00001</value>
</item>
<item>
<key>foo</key>
<value>bar</value>
</item>
</metadata>
```

List of RMC

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/s.xsl"?>
<list xmlns="http://places.mars.mipl.jpl/xml/schema">
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.1.1.1.1.1.1.1.1)?atSCLK=260.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="1" pose="1" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.1.2.1.1.1.1.1.1)?atSCLK=261.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="1" pose="2" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.1.3.1.1.1.1.1.1)?atSCLK=262.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="1" pose="3" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.2.0.1.1.1.1.1.1)" date_added="2011-09-20
14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="2" pose="0" arm="1" chimra="1" rsm="1" hga="1" drt="1" ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.2.1.1.1.1.1.1.1)?atSCLK=264.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="2" pose="1" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.2.2.1.1.1.1.1.1)" date_added="2011-09-20
14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="2" pose="2" arm="1" chimra="1" rsm="1" hga="1" drt="1" ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.2.3.1.1.1.1.1.1)?atSCLK=266.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="2" pose="3" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.3.0.1.1.1.1.1.1)?atSCLK=267.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="3" pose="0" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.3.1.1.1.1.1.1.1)?atSCLK=268.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="3" pose="1" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.3.2.1.1.1.1.1.1)?atSCLK=269.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="3" pose="2" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROVER_SUBFRAME.1.3.3.1.1.1.1.1.1)?atSCLK=270.0"
date_added="2011-09-20 14:59:51.0" frame="ROVER_SUBFRAME" site="1" drive="3" pose="3" arm="1" chimra="1" rsm="1" hga="1" drt="1"
ic="1" />
</list>
```

Solution ID List

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/patternSols.xsl"?>
<list xmlns="http://places.mars.mipl.jpl/xml/schema">
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/telemetry" id="telemetry"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/test_solution_1" id="test_solution_1"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/test_solution_2" id="test_solution_2"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/test_solution_3" id="test_solution_3"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_1" id="pose_1"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_2" id="pose_2"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_3" id="pose_3"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_4" id="pose_4"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_5" id="pose_5"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_6" id="pose_6"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_7" id="pose_7"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_8" id="pose_8"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_9" id="pose_9"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/pose_10" id="pose_10"/>
</list>
```

Solution Id Metadata

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/xml/solutionMeta.xsl"?>
<solution_id view_href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/telemetry/views"
href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/solution_id/telemetry/rmcs" id="telemetry"
xmlns="http://places.mars.mipl.jpl/xml/schema">
<origination institution="JPL" creation_date="2011-09-20" add_date="2011-09-20" method="Regression Test"
user="gangl"/>
<description>Solution added by Regression Test</description>
</solution_id>
```

Solution

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/xml/solutionXSL.xsl"?>
<solution add_date="" solution_id="telemetry" xmlns="http://places.mars.mipl.jpl/xml/schema">
<rmc site="1" drive="0" pose="0" frame="SITE"/>
```

PLACES User Guide v2.0-B

```
<offset z="12.0" y="11.0" x="10.0">
<solution_metadata>
<derived_from href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROOT.0.0.0)" site="" drive=""
pose="" frame="ROOT"/>
<reference_rmc href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROOT.0.0.0)" site="" drive=""
pose="" frame="ROOT"/>
<reference_view id="telemetry" />
</solution_metadata>
</offset>
<orientation v3="12.0" v2="11.0" v1="10.0" s="99.9">
<solution_metadata>
<derived_from href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROOT.0.0.0)" site="" drive=""
pose="" frame="ROOT"/>
<reference_rmc href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/rmc/(ROOT.0.0.0)" site="" drive=""
pose="" frame="ROOT"/>
<reference_view id="telemetry" />
</solution_metadata>
</orientation>
<origination institution="JPL" creation_date="2011-09-20" add_date="2011-09-20" method="any method" user="rgd"/>
</solution>
```

Views

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/vs.xsl"?>
<list xmlns="http://places.mars.mipl.jpl/xml/schema">
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/view/telemetry" id="telemetry"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/view/test_view_1" id="test_view_1"/>
<item href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/view/test_view_2" id="test_view_2"/>
</list>
```

View Metadata

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/xml/viewMeta.xsl"?>
<view href="https://mslopgsdev1-t.jpl.nasa.gov:8443/places/msl/dev/view/COPY/rmcs" stable="false" id="COPY of
telem" xmlns="http://places.mars.mipl.jpl/xml/schema">
<origination institution="JPL" creation_date="2011-09-20" add_date="2011-09-20" method="Regression Test"
user="rgd"/>
```

```

<description>Unit Test Clone View</description>
<parent id="telemetry" />
</view>

```

9.4. Put Operations

Put operations don't return XML like the 'Get' operations above. They do return different status codes. The following status codes can be returned:

Code	Description
201 (created)	Equivalent to 'success', request successfully completed.
400 (bad request)	Part of the query wasn't defined. Like missing information (and RMC, view name, or solution id, for example)
403 (forbidden)	User doesn't have access to the resource, possibly because the user wasn't the creator and does not belong to the group specified in the request.
404 (not found)	Part of the request was not found, for example an RMC or solution ID as part of the query does not exist.
405 (method not allowed)	The method is not allowed for a resource. For example trying to 'delete' and RMC.
409 (conflict)	Not always an error. Resource generally already exists. Trying to create a solution for an RMC that already exists.

RMC

URL	Description	Payload
\$HOST/rmc/add	Add an RMC to the system	Yes
\$HOST/rmc/RMC/metadata	Add metadata to an RMC	Yes

Saved Frame

URL	Description	Payload
\$HOST/savedFrame/FRAME/SITE,DRIVE,POSE	Associate 'FRAME' with 'SITE,DRIVE,POSE' in the database.	No

\$HOST/savedFrame/FRAME/(SITE,DRIVE,POSE)	Same as above, optional parentheses included.	No
---	---	----

View

URL	Description	Payload
\$HOST/view/	Create a view for the system	Yes
\$HOST/view/VIEW?from=VIEW[&replace=false]	Copy data from one view to another. Optional flag 'replace' is true by default. If replace is false, it will not overwrite solutions already in a view. (Will not overwrite if the view is closed).	No
\$HOST/view/VIEW?clone=VIEW	Create a view with the same entries and view specified by clone=VIEW.	No
\$HOST/view/VIEW?solution_id=SOLN_ID &RMC1=RMC[&RMC2=RMC][&type=TYPE]	Add the solution at solution_id and RMC1 to the specified view. If RMC2 is specified, it will add all solutions in the range to the view. Type is used to add only position or orientation types to the view. Both are added by default.	No
\$HOST/view/VIEW?state=close[&closeAt=RMC]	Close the view specified. If closeAt is specified, the view will be closed to the RMC specified. If closeAt is not specified, the view is permanently closed.	No

Solution/Solution_id

URL	Description	Payload
\$HOST/solution_id	Add a solution id to the system.	Yes
\$HOST/solution	Add a solution to the system	Yes

User

URL	Description	Payload
\$HOST/user?uid=USER	Add a user to the system	No

Group

URL	Description	Payload
\$HOST/group/GROUP	Add a group to the system.	No
\$HOST/group/GROUP?uid=USER	Add a user to the specified group	No

Admin Functions

URL	Description	Payload
\$HOST/view/VIEW?state=open	Opens a closed view. If closed at an RMC, will set the RMC to null.	No
\$HOST/view/VIEW?copy=VIEW[&levels=LEVELS]	Copy a view and it's parent view up to LEVELS deep. Copy at levels=0 is the same as clone. Levels is 0 by default.	No
[delete]\$HOST/view/VIEW	Delete a view	No
[delete]\$HOST/view/VIEW&rmc=RMC&solution_id=SOLN_ID	Delete a solution from a view. This should only be done carefully and with the agreement of most users of the system.	No
[delete]\$HOST/group/GROUP?uid=USER	Remove a user from a group	No
[delete]\$HOST/solution/SOLN_ID/RMC	Delete a solution	No

9.5. Put XML Payloads

RMC

```
<rmc frame="ROVER_SUBFRAME" site="1" drive="2" pose="3" arm="4" chimra="5" drill="6" rsm="7" hga="8"
drt="9" ic="10" actual_sclk="1110.001" />
```

RMC Metadata

```
<metadata>
  <item>
    <key>key</key>
    <value>value</value>
  </item>
  <item>
    <key>foo</key>
    <value>bar</value>
  </item>
</metadata>
```

View

```
<view id="test_view_3" group="tactical" > <!--group used to set permissions for write using the view -->
<parent>telemetry</parent>
  <origination institution="JPL" creation_date="2011-09-12" add_date="2011-09-12" method="Regression Test"/>
  <description>Unit Test Create View</description>
</view>
```

Solution_Id

```
<solution_id id="test_telemetry" group="tactical">
  <origination institution="JPL" method="Regression Test" user="gangl"/>
  <description>Solution added by Regression Test</description>
</solution_id>
```

Solution

For solutions, you can have only offset, only orientation, or both.

```
<solution solution_id="test_solution_3">
  <rmc frame="ROVER" site="1" drive="2" pose="0" />
  <offset z="3.0" y="2.0" x="1.0">
```

```
<solution_metadata>
  <reference_rmc site="1" drive="0" pose="0" frame="SITE"/>
  <reference_view>telemetry</reference_view>
  <derived_from site="1" drive="1" frame="ROVER"/>
</solution_metadata>
</offset>
<orientation v1="3.0" v2="2.0" v3="1.0" s="5.0">
  <solution_metadata>
    <reference_rmc site="1" drive="0" pose="0" frame="SITE"/>
    <reference_view>telemetry</reference_view>
    <derived_from site="1" drive="0" pose="0" frame="SITE"/>
  </solution_metadata>
</orientation>
<origination user="gangl" method="DB" institution="JPL"/>
</solution>
```

10. Use Cases

This section contains use cases - example scenarios with a realistic task to perform and how you would go about accomplishing the task using PLACES.

10.1. *What was the rover's position at RMC (5,12)?*

Use the “tactical” view (a reasonable default when nothing else is specified), Site 5, and the primary query.

```
$HOST/query/primary/tactical?from=rover(5,12)&to=site(5)
```

Note that the default is to return position only.

10.2. *How does Joe think the rover was oriented at RMC (5,12)?*

Similar to above, except we use a different view, e.g. “joe_localizations”. Also, we ask for orientation instead of position.

```
$HOST/query/primary/joe_localizations?from=rover(5,12)&to=site(5)&return=orientation
```

Returned value is a quaternion, indicating the rotation of the Rover frame with respect to the Site.

10.3. *What Views have you created?*

Assume you use your username in your view names. Note that wildcards are implicit on both sides of the pattern, so this will find any views containing the given string. Asterisks (*) can be embedded in the string for more complex searches.

```
$HOST/views/rgd
```

If you did not use your username in the view names, the question can still be answered with more work. Leave off the pattern in order to get a list of all views:

```
$HOST/views
```

Then loop through the returned list. Each item will have a link (URL), which is the proper query to find that view's metadata. Follow the link to get the view metadata, and examine the “origination” field to look for your username.

10.4. *Creating a Traverse Map*

A traverse map consists of an orbital image with the rover locations plotted on it. To create one of these, first pick a View to use (e.g. best_tactical_nnnn or a science view). Then pick the orbital image to use. We assume here the orbital image is already mapped to PLACES, making the process easy. If not, you can pick some other orbital image, or the landing site frame, as the reference and transform the coordinates returned by PLACES to your chosen image yourself.

Loop over the sites or (site,drive) locations of interest. For each one, execute the primary query, requesting the position in the selected Orbital image. For Sites:

```
$HOST/query/primary/best_tactical_0325?from=site(x)&to=orbital(z)
```

For non-Site positions:

```
$HOST/query/primary/best_tactical_0325?from=rover(x,y)&to=orbital(z)
```

Gather the returned locations and plot them on the orbital image.

Note that you could ask PLACES for a list of all RMC's it knows about within a given range, and use that to control the loop. For example, to get all the RMC's in the view between sites 5 and 12:

```
$HOST/view/best_tactical_0325/rmcs/site(5),site(12)
```

You could even extend this query to include dynamic RMC's and get a complete traverse history within the given range.

TBD: URL syntax for deep listing

10.5. Mosaic Two Images from Different Positions

Say you have two images, image 1 taken at RMC (5,12), and image 2 taken at RMC (5,16), and you want to mosaic them together. That requires knowing the precise relationship between the two rover positions. Start by picking a common frame for the mosaic. This would typically be the Site frame, or the Rover frame from one of the images. For the first example, we'll assume the Site frame. As usual, pick a View to use, in this case "best_tactical_^", which is the latest and greatest. Execute the primary query for each image, converting to the common Site frame:

```
$HOST/query/primary/best_tactical_^?from=rover(5,12)&to=site(5)&return=both
```

```
$HOST/query/primary/best_tactical_^?from=rover(5,16)&to=site(5)&return=both
```

This returns both the position and orientation of the rover at both locations. Use these poses to create the mosaic.

Now say the common frame is that of image 1. In that case there's only a single query, getting image 2's position in terms of image 1:

```
$HOST/query/primary/best_tactical_^?from=rover(5,16)&to=rover(5,12)&return=both
```

Image 2 can now be projected to match image 1, and mosaicking can proceed.

10.6. Update Target with New Localizations

For this scenario, assume a target has already been selected by MSLICE or other tool and put in the target database (which is unrelated to PLACES). However, it was entered several days ago during

tactical ops and there are better rover localizations in the most recent tactical view (best_tactical_^) now to make use of.

First, retrieve the target XYZ along with the frame and view from the target database. Let's say the target was at xyz=6.3, 5.01, -0.2), frame=site(5), and view=tactical. Execute the transform query, converting the given point/frame/view to the same frame in the new view:

```
$HOST/query/translate/point/(6.3,5.01,-0.2)/tactical?from=site(5)&to=site(5)&inView=best_tactical_^&return=position
```

Either use new target position, or store back in target database with the new view name. If you used "best_tactical_^", make sure to pull the actual view name from the translate response; do not put ^ in the target database.

10.7. Point the Camera at a Target

Assume you want to point the camera at a target that was defined in a previous Rover frame. You must decide what frame to use to issue the command: Site, Rover, or a Saved frame. For this example we'll assume Saved 5; if you're not using the Saved frame then just omit the saved frame lookup. First, execute the Saved Frame lookup to get the equivalent RMC:

```
$HOST/savedFrame/5
```

For this example, say it returns (9,16,2). The saved-frame lookup returns only the first three RMC elements because by definition, the rover's idea of the frame is the same no matter what the mechanism indices are. Retrieve the target from the target database as in the previous example. Let's say the target is xyz=-42.31, 12.225, 0.02), frame=rover(9,14), and view=best_tactical_0141. The transform query would be:

```
$HOST/query/translate/point/(-42.31,12.225,0.02)/best_tactical_0141?from=rover(9,14)&to=rover(9,16,2)&inView=telemetry
```

Write the command using returned XYZ and frame RNAV_SAVED 5. You could also say "to=site(9)" in the query and then use SITE_SAVED_5, or "to=local_level(9,16,2,4,10,8,10,0,0,4)" and use LL_SAVED 5.

10.8. Create New Localization in Existing View

You have run your favorite localization process and now you want to add it to PLACES. The first step is to create the solution ID if you haven't already. It's okay to do this even if it already exists, you'll simply get an HTTP status back indicating that. First you need an XML file, which defines all the metadata for this solution ID. Note that this describes the solution ID itself - not the actual solution.

```
<solution_id id="rgd_solutions" >
  <origination institution="JPL" creation_date="2011-09-21"
    add_date="2011-09-21" method="How I created these solutions"
    user="rgd"/>
  <description>Solution added by RGD</description>
</solution_id>
```

Then create the solution ID, providing the XML file as the payload for the HTTP Put operation.

```
PUT: $HOST/solution_id
```

Now, it's time to create the solution. First, decide on appropriate derived-from and reference frames. In almost all cases the reference frame will be the enclosing site. Determining the proper Derived-from frame is trickier; see Section 5.6. In this case, let's say the solution, which is for RMC (7,22), was derived via image analysis by comparing it with an image taken at (7,16). Therefore the Reference frame is Site(7) and the Derived-from is Local_Level(7,22). Because you want to take advantage of other localizations in the same View, the reference view is the one we are putting this into, "rgd_updates". So the XML file for this solution looks like this:

```
<solution solution_id="rgd_solutions">
  <rmc site="7" drive="22" frame="ROVER"/>
  <offset z="12.38" y="11.22" x="0.01">
    <solution_metadata>
      <derived_from site="7" drive="16" frame="LOCAL_LEVEL"/>
      <reference_rmc site="7" frame="SITE"/>
      <reference_view>rgd_updates</reference_view>
    </solution_metadata>
  </offset>
</solution>
```

Because there is only an <offset> section, this sets position and not orientation. Create the solution, using the XML file as the HTTP Put payload:

```
PUT: $HOST/solution
```

Finally, add the solution to the view:

```
PUT: $HOST/view/rgd_updates?solution_id=rgd_solutions&rmc=rover(7,22)
```

In this case, there is no XML payload to go along with the Put.

10.9. Create New View with an Update

Let's say you like most of Joe's solutions, but you think he screwed up at (7,22) and you want to change it. Of course Joe thinks he's right. So you need another View to hold your update. Meanwhile Joe can continue to use his View with no interference from you.

First, create a new view, using Joe's view as a parent. The XML looks like this:

```
<view id="joe_with_rgd_fixes" parent="joes_view">
  <origination institution="JPL" method="new Method"/>
  <description>Fixes to Joe's localizations</description>
</view>
```

The parent attribute says that this view will look like joes_view, except for whatever changes are added. Create the view using the XML payload:

```
PUT: $HOST/view
```

Now, add your solution (created in the previous example) in the new view:

```
PUT:  $HOST/view/joe_with_rgd_fixes?solution_id=rgd_solutions&rmc=rover(7,22)
```

The last step is optional: you can close your view to that point. This prevents further change before that point, but provides immutability to queries.

```
PUT:  $HOST/view/joe_with_rgd_fixes?close=true&closeAt=rover(7,22)
```

Once again, no payload is needed for this Put.

11. Glossary

attitude: Synonymous with “orientation”.

chain: A set of links between Derived-from frames that implement the system by which one localization can affect other downstream ones.

clone: A copy of a view; shares the same parent but has a copy of the contents.

closed view: A view that cannot be modified. Views are closed up to a certain point, identified with an RMC value.

deep copy: An expensive copy of a view, which may include several levels of parents.

Derived-from frame: The frame from which a solution was derived. Need not be the same as the Reference frame.

Drive: Second index of the RMC; indicates when the rover is driving (odd) or stationary (even).

dynamic RMC: An RMC with an odd value for Drive or Pose; indicates the rover is in motion or updating pose knowledge.

estimate: Synonymous with “solution”.

frame: An instance of a coordinate system. Combines the type of frame (Site, Rover, Local_Level, Orbital) with an instance index (RMC, SCLK, or counter). The frame has a single but unknowable true pose; solutions provide estimates of that true pose.

groups: A mechanism to control access to views and solution IDs.

localization: The process of determining where the rover is and how it is oriented at any given time.

Local Level frame: A combination of Rover frame position with Site frame orientation.

mechanism RMC: The last 7 values of RMC (Arm, CHIMRA, Drill, RSM, HGA, DRT, IC), which indicate motions of mechanisms.

metadata: Descriptive information attached to database entries (views, solution ID’s, etc) that plays no part in coordinate transformations.

mobility RMC: The first 3 values of RMC (Site, Drive, Pose), which have special meaning for mobility and the PLACES database.

open view: A view that can be modified. See closed view.

orientation: How the rover or a frame is oriented in space. Specifically, a rotation relative to the reference frame.

orbital images: Maps used as an aid to localization and areoreferencing.

PLACES: Database storing localization solutions for all users across the project.

pose: The combination of position and orientation of the rover or a frame. When capitalized, indicates the 3rd index of the RMC.

position: Where the rover or a frame is located. Specifically, a translation or offset in Cartesian XYZ space from the reference frame.

quaternion: A 4-element vector used to represent a rotation in 3-dimensional space. Used to represent orientation of the rover in PLACES.

Reference frame: The frame in which coordinates (offsets or rotations) are expressed. See also Derived-from frame.

ReST web service: A mechanism for implementing web services using simple URL's and XML files.

Root: The frame at the root of the coordinate frame tree, from which all others are derived. Defined to be the same as the landing site.

Rover Frame: A coordinate frame attached to the rover, with +X pointing forward and +Z toward the belly.

RMC: A set of 10 indices used as a clock to track rover motion. Used as the instance identifier for most Rover frames. The first three are the mobility RMC, the rest are the mechanism RMC.

saved frame: A mechanism used onboard to save and reuse some frames after the rover moves from that location.

site: An area of operations; establishes a coordinate system origin for use in local operations.

Site Frame: A coordinate frame attached to the ground, with +X pointing north and +Z down according to the local gravity vector.

sol: One Martian day.

solution: One specific position or orientation (out of possibly many) for a given RMC. Represents an individual estimate of the unknown true value of position or orientation.

solution ID: A string used to identify a set of solutions.

stable RMC: An RMC with even values for the Drive and Pose indices; indicates the rover is not intentionally moving or updating its pose knowledge.

subframe: A database implementation detail; combines dynamic RMC and unsolicited move frames in a single term.

tactical: View in which day-to-day operations are performed.

telemetry: View containing only values obtained from the rover; it is the root of the view tree.

unit vector: A 3-element vector with magnitude 1.0, used to indicate a direction in space (such as the direction a camera is pointing) independent of position.

unsolicited move: An unexpected motion of the rover, such as slipping off a rock due to a wind gust.

view: A named entity which provides a single solution for each RMC. Used to differentiate different sets of solutions.

visual odometry (visodom): A mechanism to refine pose onboard by analyzing images before and after a drive.

XYZ: Cartesian coordinates used to express an offset or position.

12. Acronym List

CHIMRA: Collection and Handling for Interior Martian Rock Analysis

DRT: Dust Removal Tool

EVR: Event Verification Record

FSW: Flight Software

HGA: High Gain Antenna

HTTP: HyperText Transfer Protocol

Hz: Hertz

IC: Inlet Cover

IDD: Instrument Deployment Device (MER)

IDPH: Image (or Instrument) Data Product Header

IMU: Inertial Measurement Unit

JPL: Jet Propulsion Laboratory

LDAP: Lightweight Directory Access Protocol

MER: Mars Exploration Rover

MSL: Mars Science Laboratory

PDS: Planetary Data System

PLACES: Position Localization and Attitude Correction Estimate Storage

PMA: Pancam Mast Assembly (MER)

ReST: Representational State Transfer

RMC: Rover Motion Counter

RSM: Remote Sensing Mast

SCLK: Spacecraft Clock

URL: Uniform Resource Locator

visodom: Visual Odometry

XML: Extensible Markup Language

13. TBD list

Specific contents of XML summary files and operational decisions as to what to extract and when.

Specific telemetry sources.

Should Solutions include an error metric? (for documentation only, not used by PLACES)

Revisit the subframe writeup. The description would only allow unsolicited moves to be subframes since with dynamic there is no stable parent. Which might be okay, except how do you get a list of all dynamic frames for a given RMC triplet base? Could be returning a list of unsolicited frames is what's really needed, with a separate query to list all dynamics?